

# Append HMI Studio

## User Manual

Designing, testing and deploying HMI applications,
illustrated with the LiquidWeighHMI example

Version 1.4 · October 2026
Append Automation

# Contents

1 Introduction....5
1.1 Building applications with an AI agent....6
1.2 How an HMI application is organized....7
2 The Studio window....8
2.1 The HMI menu....9
3 Application settings....10
4 Devices....12
5 Tags....13
5.1 Tag fields....15
5.2 Alarms on a tag....15
5.3 Renaming, and finding and replacing tag references....16
6 Screens and windows....17
6.1 Window properties....17
6.2 The header, the navigation bar and the screens....19
7 Drawing objects....20
8 Animation links....21
8.1 A valve: status color, fault blink and faceplate....23
8.2 The weigh hopper: percent fill....24
8.3 Hopper pressure: value display and alarm colors....26
8.4 Setpoints: user input....27
8.5 Commands: pushbutton and enable....28
9 Expressions and scripts....29
9.1 Window and application scripts....30
9.2 Action scripts....34
9.3 Function reference....35
9.4 Indirect tags: one faceplate for every device....40
9.5 Opening windows from scripts: ShowWindow....42
9.6 Saving the screen as a PDF: ScreenToPDF....45
9.7 Local variables: DIM....46

9.8 QuickFunctions....47
10 Alarms and security....49
10.1 Alarms....49
10.2 Users and access levels....50
11 Recipes....51
11.1 Recipe books....52
11.2 The Recipes screen....54
11.3 The Recipe List object....56
11.4 Selecting a recipe in a window....57
11.5 When a recipe function fails....58
11.6 Where recipes are kept....59
12 Trends....59
12.1 The Trend object....59
12.2 Pens and their ranges....61
12.3 The Trends screen....62
12.4 Scrolling through the history....63
12.5 Recording and where the history is kept....64
13 Databases....65
13.1 Data sources....66
13.2 Bind lists....67
13.3 Table templates and queries....68
13.4 The script functions....69
13.5 The batch log in LiquidWeighHMI....71
13.6 Where the rows are kept....72
14 List boxes....73
14.1 The List Box object....73
14.2 Filling and working the list from scripts....76
14.3 The list belongs to the application, not to a window....77
14.4 A list filled from a database....77
15 Testing....78

15.1 Validate....78
15.2 Run....79
16 Deploying....83
16.1 Publish....84
16.2 Append HMI Desktop....85
16.3 Append HMI Web....85
17 Tips and troubleshooting....87
Appendix A Command-line automation....88

# 1 Introduction

Append HMI Studio designs operator screens (HMIs) for PLCs, runs them against live equipment and publishes them for the plant floor. This manual shows how to build an HMI application with it, step by step, using a complete, working example: **LiquidWeighHMI**, the operator interface of a liquid weigh-up system that meters three plasticizers into a weigh hopper and discharges each batch to a mixer.

What sets Append HMI Studio apart is that it is made to be used by **AI agents** as well as by people. An agent can build a complete, good-looking and working HMI application from the documents a project already has: the functional description or control narrative, the PLC program and the electrical drawings. The example in this manual was built that way: an AI agent wrote LiquidWeighHMI from scratch, and wrote its PLC program too (section 1.1).

The example is included with Append HMI Studio as `examples/LiquidWeighHMI.ahmi`. Open it with **File > Open** and follow along; every screenshot in this manual was taken from it, the run-mode ones with the application connected to its ControlLogix controller.

*Table 1. What the LiquidWeighHMI example contains*

| Part | In the example |
|---|---|
| **Target screen** | 1920 × 1080, full screen |
| **PLC** | One ControlLogix controller (EtherNet/IP), 419 I/O tags |
| **Screens** | Process, Batch, Supply, Devices, Setpoints, Recipes, Trends, Batch History, Alarm History and IO Sim |
| **Popup windows** | One faceplate for all 11 valves and 3 pumps, through indirect tags, and 4 confirmation dialogs |
| **Alarms** | 34 alarmed tags, an Active Alarms list on every screen and an Alarm History screen |
| **Security** | Three users (Operator, Maintenance, Engineer) with access levels |
| **Recipes** | A recipe book of plasticizer blends with five starting recipes, and a Recipes screen to select, edit, save and download them |
| **Database** | Every completed batch written to a SQLite file in Documents, and a Batch History screen that reads them back (chapter 13) |
| **Built by** | An AI agent, from the control narrative, the I/O list and the electrical drawings, after it had written the PLC program (section 1.1) |
| **Scripts** | Window scripts that turn the sequence step number into text, and action scripts that call the login, recipe, indirect tag, ShowWindow and ScreenToPDF functions (a PRINT SCREEN button on every screen) |

## 1.1 Building applications with an AI agent

Most HMI packages can only be driven through their own screens, one dialog at a time. Append HMI Studio also has a complete command-line interface and documentation written for AI agents, so an agent such as Claude Code can do the whole job: read the project documents, design the screens, build and validate the project, look at its own work, correct it and publish the installer.

**LiquidWeighHMI was written from scratch by an AI agent.** It was given three documents and no HMI to start from:

- **The control narrative:** a short description of the liquid weigh-up system and how it should operate.
- **The I/O list:** a spreadsheet of the field devices and their I/O points.
- **The electrical drawings:** a scanned PDF with no text layer, which the agent read as images, including the I/O and P&ID sheets.

There was no PLC program either: the agent wrote that first. From the same documents it wrote the ControlLogix program: 13 ladder routines for the sequence, recipe, interlocks, valves, pumps, alarms and a simulation of the process. It then checked the program with a static checker and a ladder simulator of its own, which ran full batches and scenario tests and found a real bug before the program ever reached a controller. Along the way it found and reported places where the I/O list disagreed with the drawings. Only then did it design the HMI, with every I/O tag addressed to the program's real tag paths.

The result is the application this manual describes: 419 PLC tags, nine screens, a faceplate shared by 14 devices, alarms, users with access levels, recipes and trends, in a consistent visual style. The engineer's part was to supply the documents, review the screens and ask for changes, mostly fine points of layout such as how pipes meet the valve symbols. The same agent later turned the PLC program into the application scripts of **LiquidWeighHMISim**, the version of the example that runs without a PLC (section 9.1).

*Table 2. How an AI agent builds an application*

| Step | What the agent does |
|---|---|
| **Understand** | Reads the functional description, the PLC program (or writes it) and the drawings, and lists the tags, devices and screens |
| **Describe** | Writes the application as a JSON spec, usually generated by a small script so every screen shares the same building blocks |
| **Build** | --hmi-build turns the spec into a project; spec errors and PROBLEM lines name exactly what to fix |
| **Check** | --hmi-check validates every expression, script, window reference and PLC address |
| **Look** | --hmi-render draws every screen to PNG, so the agent can see its own layout and correct it |
| **Change** | --hmi-dump writes a project, including the engineer's own edits in the Studio, back out as JSON without losing anything |
| **Deliver** | --hmi-publish builds the Windows installer for the operator station |

**Documentation ready for agents.** Everything an agent needs is written down in a form it can use without guessing, so it can produce good-looking, working applications with very little intervention:

*Table 3. Documentation for AI agents*

| Document | What it gives the agent |
|---|---|
| **Append HMI Studio skill** | A Claude Code skill (`.claude/skills/append-hmi-studio`): the workflow, the rules that matter and good screen practice: palette, text sizes, navigation and alarms |
| **HMI_AUTOMATION.md** | Every command and exit code, every field of the JSON spec, every object and animation link type, and the expression and script syntax |
| **HMI_ALARMS.md, HMI_SECURITY.md, HMI_RECIPES.md, HMI_INDIRECT.md** | How alarms, users and access levels, recipes and indirect tags behave, with their spec fields and script functions |
| **example-spec.json** | A complete small application to start from |

Appendix A lists the command-line modes. A person can open anything an agent builds in the Studio and carry on by hand, and the agent picks up those changes the next time it dumps the project.

## 1.2 How an HMI application is organized

An Append HMI Studio project (`.ahmi` file) holds everything the operator station needs:

- **Devices**: the PLCs the application talks to (Allen-Bradley Logix, SLC 500 and MicroLogix, Modbus TCP, or the built-in simulator).
- **Tags**: named values, either read from and written to a PLC (I/O tags) or kept by the HMI itself (memory tags). Tags carry engineering units, ranges and alarm limits.
- **Windows**: each page of the drawing is a window of the application, either a full screen or a popup such as a faceplate.
- **Objects and animation links**: the shapes on the screens, and the links that tie them to tags: colors that follow a state, levels that fill, values that display, buttons that write.
- **Application settings**: the target screen resolution, the windows that open at start-up, how the published application runs, and its users.

The usual order of work, which this manual follows, is: application settings, devices, tags, screens and windows, animation, alarms and security, then testing with Run and deploying.

# 2 The Studio window

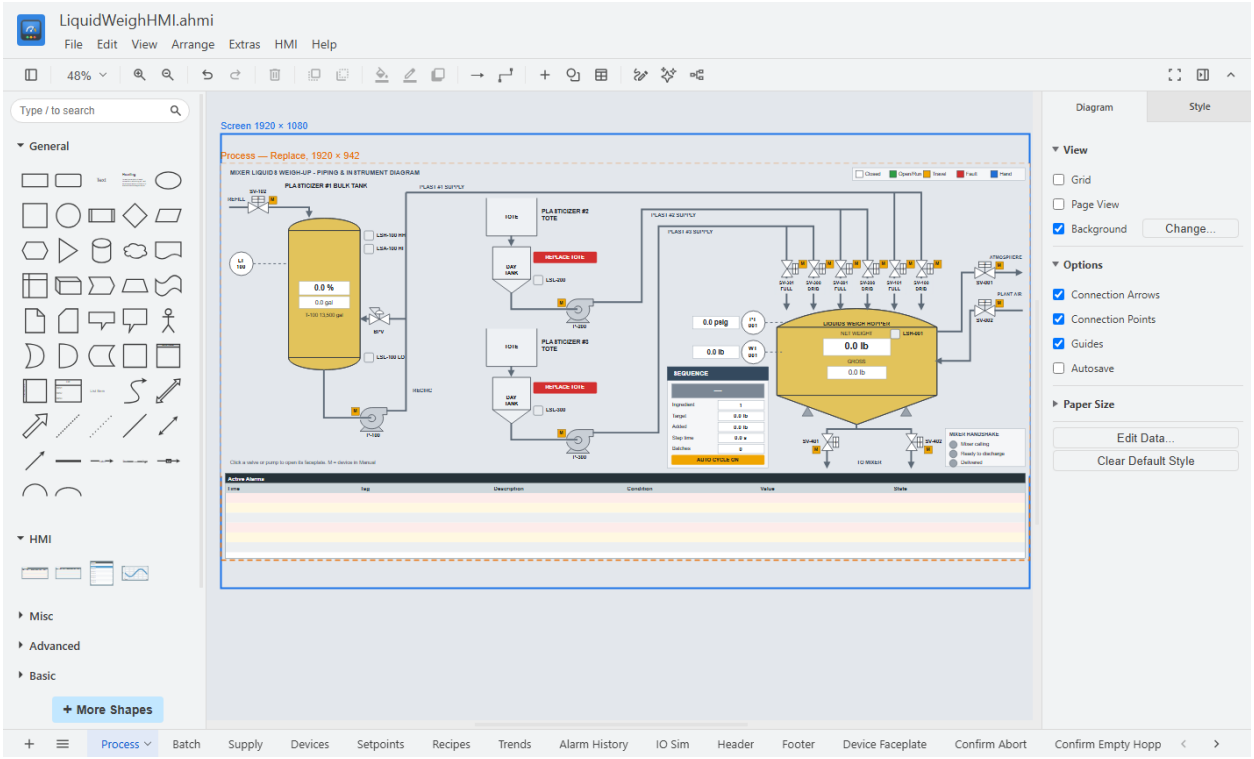



*Figure 1. Append HMI Studio with the LiquidWeighHMI Process screen open*

*Table 4. Parts of the Studio window*

| Area | What it is for |
|---|---|
| **Menu bar** | File, Edit, View, Arrange and Extras from the drawing editor, plus the HMI menu for everything specific to HMI applications |
| **Toolbar** | Zoom, undo, delete, arrange, and quick access to fill, line and shadow |
| **Shapes** | Shape libraries (General, Misc, Advanced and more under More Shapes, including the P&ID libraries) and the HMI palette with the Alarm List, Alarm History, Recipe List and Trend objects |
| **Canvas** | The page being edited. The dashed Screen frame shows the target screen (1920 × 1080 here) |
| **Format panel** | Style, Text and Arrange for the selected object, and the Animation tab for its links |
| **Page tabs** | One tab per window of the application; + adds one |

## 2.1 The HMI menu

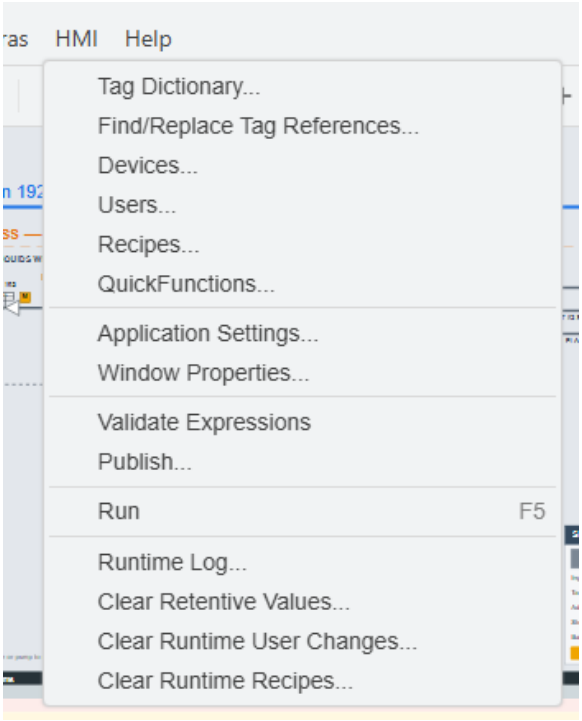



*Figure 2. The HMI menu*

*Table 5. HMI menu commands*

| Command | Use |
|---|---|
| **Tag Dictionary...** | Create and edit tags (chapter 5) |
| **Devices...** | Configure the PLCs (chapter 4) |
| **Users...** | Users, passwords and access levels (chapter 10) |
| **Recipes...** | Recipe books: the tags a recipe holds and the starting recipes (chapter 11) |
| **Application Settings...** | Target screen, start-up windows, runtime, security and application scripts (chapter 3) |
| **Window Properties...** | Type, position, size and scripts of the current window (chapter 6) |
| **Validate Expressions** | Check every link, script and address for errors (chapter 15) |
| **Publish...** | Build a Windows installer of the application (chapter 16) |
| **Run / Stop (F5)** | Run the application in the Studio against live or simulated data (chapter 15) |
| **Runtime Log...** | Messages from the last runs: connections, writes, script errors |
| **Clear Retentive Values...** | Forget the saved values of retentive tags on this computer |
| **Clear Runtime User Changes...** | Forget user changes made while running on this computer |
| **Clear Runtime Recipes...** | Forget the recipes saved while running on this computer |

# 3 Application settings

Start a new application by setting its target screen: the resolution of the operator station it will run on. Open **HMI > Application Settings**.

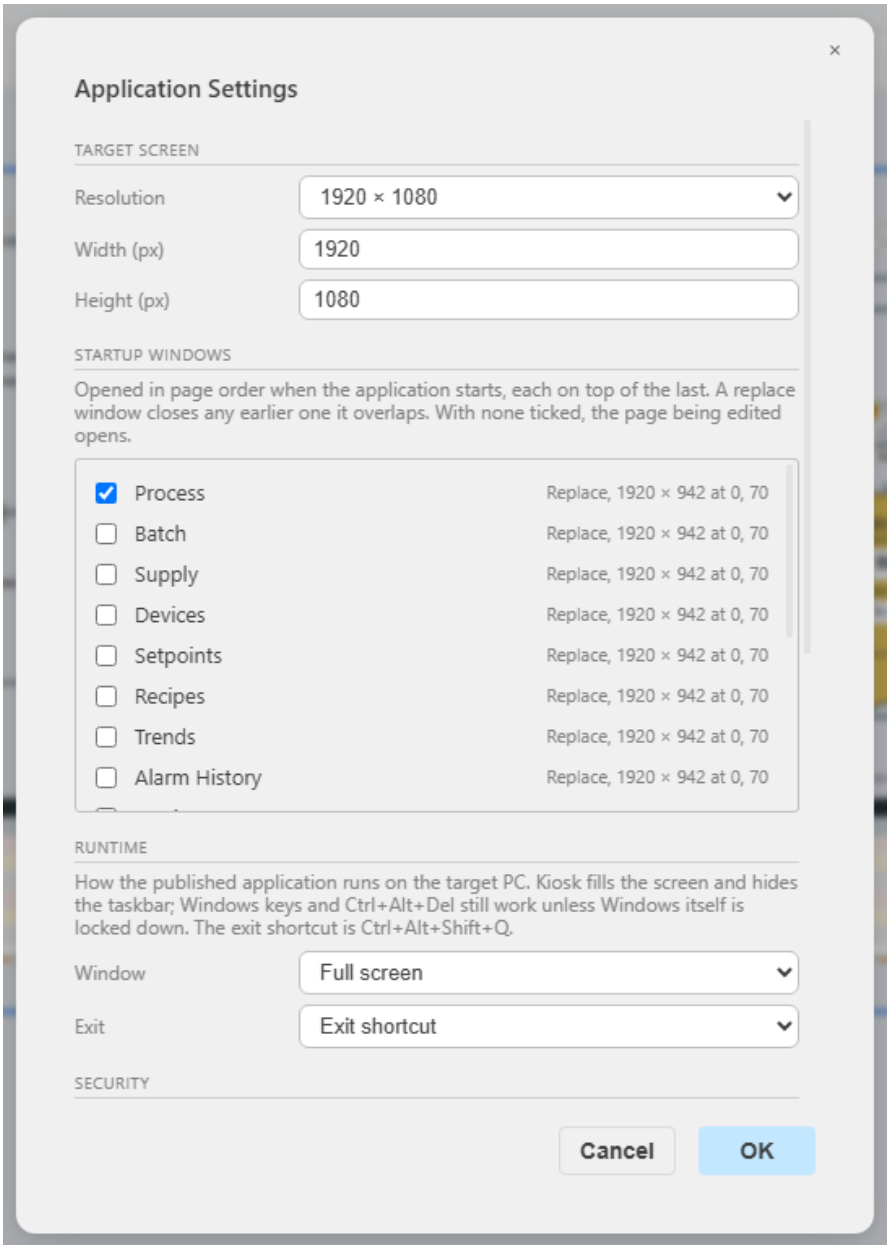



*Figure 3. Application Settings: target screen and start-up windows*

- **Resolution**: choose a common size or type the width and height. LiquidWeighHMI targets a 1920 × 1080 panel PC. The dashed Screen frame on every page follows this size.

- **Startup windows**: the windows that open when the application starts, in page order, each with its type, size and position. LiquidWeighHMI opens the Process screen and the Header and Footer windows that go with every screen (section 6.2). With none ticked, the page being edited opens.

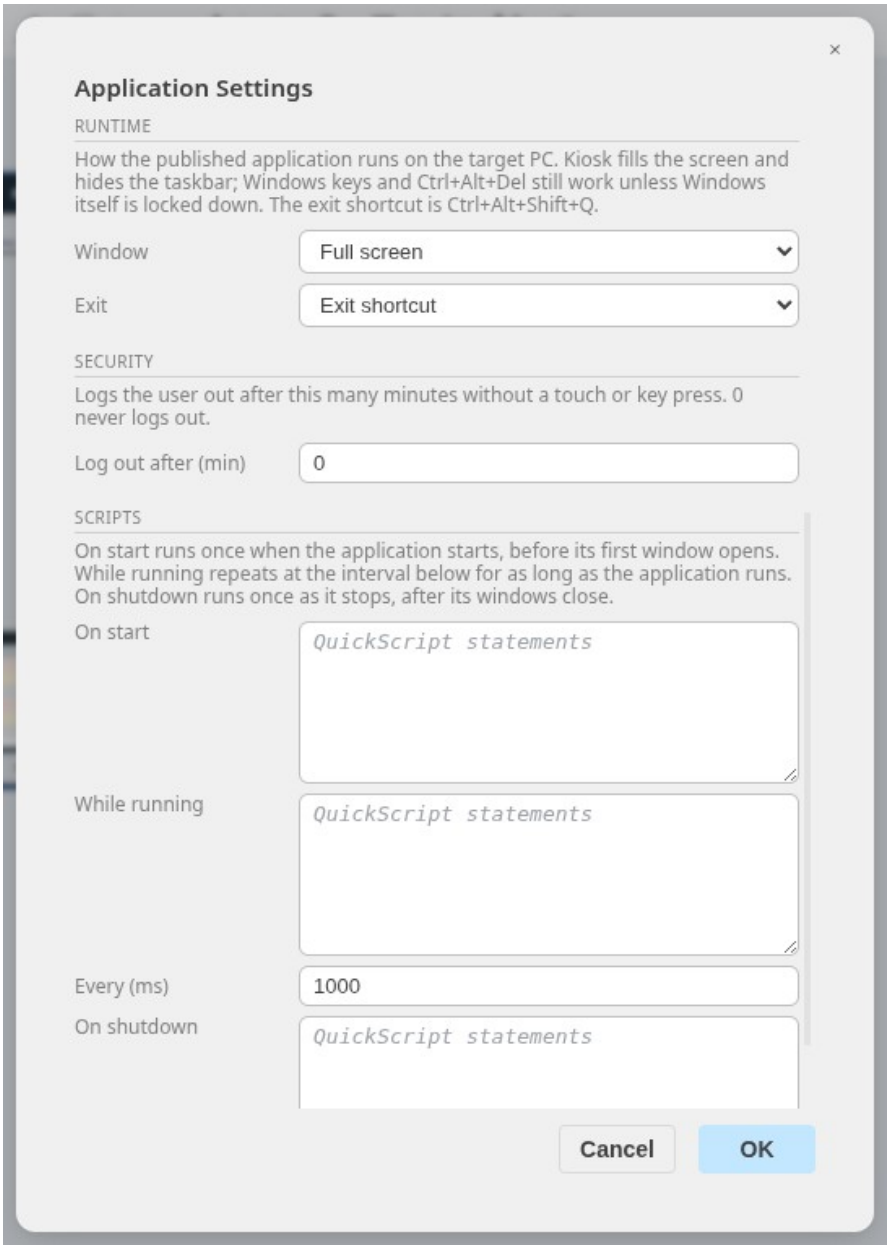



*Figure 4. Application Settings: runtime, security and scripts*

- **Window**: how a published application runs on the target PC: **Kiosk** (full screen, locked, taskbar hidden), **Full screen**, or a **window** at the target resolution. LiquidWeighHMI uses Full screen.
- **Exit**: the exit shortcut Ctrl+Alt+Shift+Q; the shortcut followed by a password; or never (the PC must be shut down to stop it).
- **Log out after (min)**: logs the operator out after this many minutes without a touch or key press. 0 never logs out.

- **Scripts**: the application scripts, which run whichever windows are open: **On start**, **While running** (every n ms) and **On shutdown** (section 9.1).

# 4 Devices

Devices are the PLCs the application communicates with. Open **HMI > Devices**, click **Add**, and fill in the connection. LiquidWeighHMI has one device, **LiquidWeighPLC**, a ControlLogix controller.

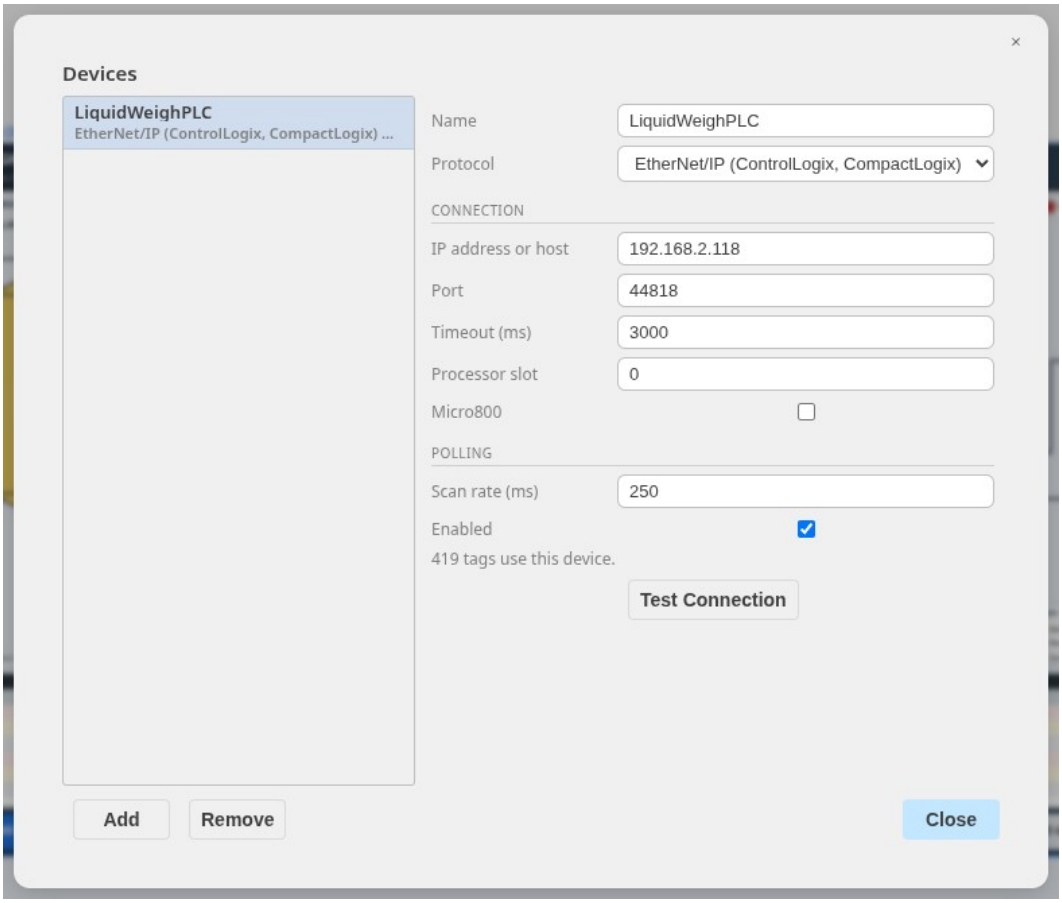



*Figure 5. The Devices dialog with the LiquidWeighPLC controller*

- **Protocol**: the communication driver (Table 4).
- **IP address or host**, **Port** (44818 for EtherNet/IP, 502 for Modbus TCP) and **Timeout**.
- **Processor slot**: the controller's slot in the chassis (0 for a CompactLogix).
- **Scan rate**: how often subscribed tags are read, 250 ms here.
- **Test Connection** checks that the PLC answers before any screen is built.

*Table 6. Protocols and address examples*

| Protocol | Controllers | Address examples |
|---|---|---|
| **EtherNet/IP (Logix)** | ControlLogix, CompactLogix, Micro800 | PT_001.Value<br>Program:Main.Pump.Run<br>Recipe[3]<br>Status.5 |
| **PCCC (SLC)** | SLC 5/05, MicroLogix | N7:0<br>B3:1/4<br>F8:2<br>T4:0.ACC |
| **Modbus TCP** | Any Modbus TCP device | HR:0<br>HR:10:FLOAT<br>CO:5<br>IR:3 |
| **Simulator** | None (built in) | Simulated values for testing |

**Note:** Communication runs through a small companion program, the hmi-comms server, which the Studio starts by itself. Device states and errors appear in HMI > Runtime Log.

# 5 Tags

Tags are the named values the screens show and change. Open **HMI > Tag Dictionary**. The list on the left shows every tag with its type; type in **Filter** to find one. Select a tag to edit it, or choose a type under **New tag…** to create one.

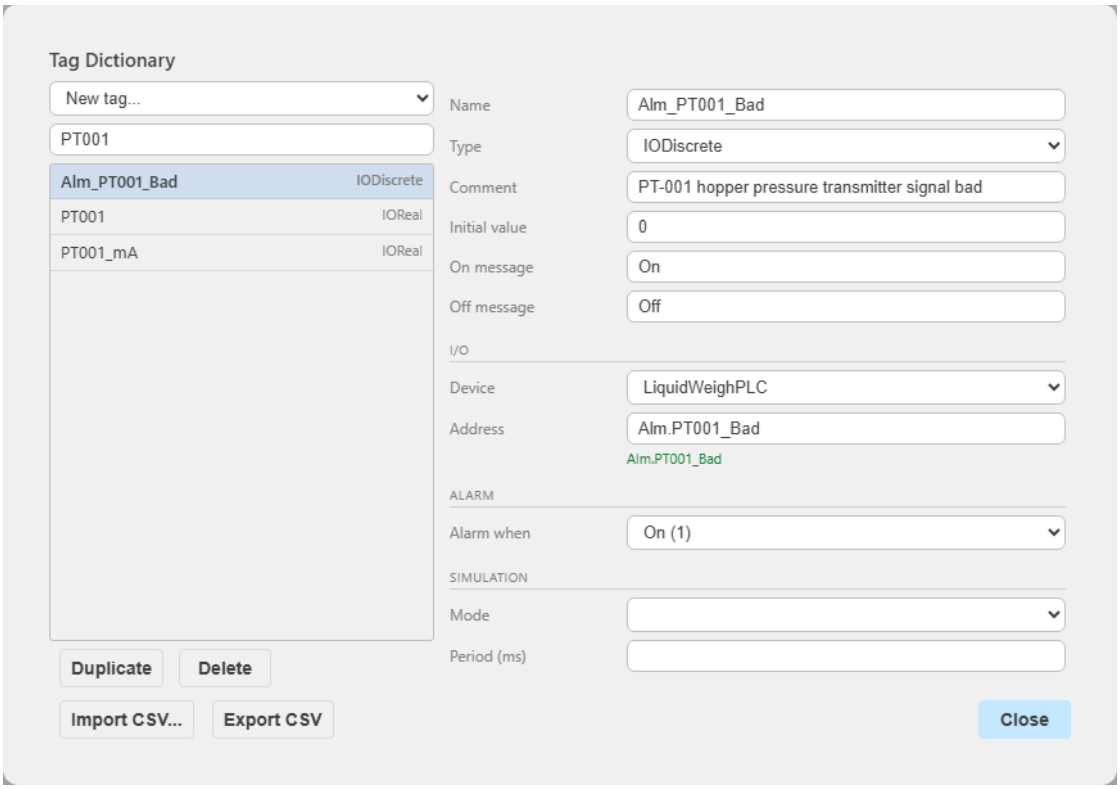



*Figure 6. The Tag Dictionary with PT001, the weigh hopper pressure*

*Table 7. Tag types*

| Type | Holds | LiquidWeighHMI example |
|---|---|---|
| **IODiscrete** | On/off from a PLC | SV001_Opened (valve open limit) |
| **IOInteger** | Whole number from a PLC | SV001_Status (0 closed, 1 open, 2 travel, 3 fault) |
| **IOReal** | Real number from a PLC | PT001 (hopper pressure, psig) |
| **IOMessage** | Text from a PLC | Recipe or product names |
| **MemoryDiscrete, MemoryInteger, MemoryReal, MemoryMessage** | A value kept by the HMI | StepText (sequence step description) |
| **IndirectDiscrete, IndirectAnalog, IndirectMessage** | No value of its own: stands for the tag a script links it to with LinkIndirectTag | FP_Status: the status of the device the faceplate shows |

## 5.1 Tag fields

*Table 8. Tag fields*

| Field | Meaning |
|---|---|
| **Name** | Letters, digits and underscores; not case-sensitive. LiquidWeighHMI names tags after the device and signal, such as SV001_Status and PT001 |
| **Comment** | A description. For an alarmed tag it is the alarm text operators see |
| **Engineering units, Minimum EU, Maximum EU** | Units and range, used by fills, bars and value displays |
| **Initial value** | Starting value of a memory tag |
| **Device, Address** | For an I/O tag: the PLC and the address in its syntax. The address is checked as you type (green when valid) |
| **Scale raw values** | Maps a raw integer range (for example 0 to 4095) to the engineering range |
| **Retentive** | For a memory tag: keeps its last value between runs |
| **Simulation** | A simulated value (sine, ramp, random, toggle) for testing without the PLC |

## 5.2 Alarms on a tag

Scroll down to **Alarms** to give a tag alarm limits. PT001 alarms at High 70 psig and HiHi 85 psig, with a deadband of 2 so the alarm clears only below 68 psig. A discrete tag instead has **Alarm when** On or Off: SV001_Fault alarms when it is on.

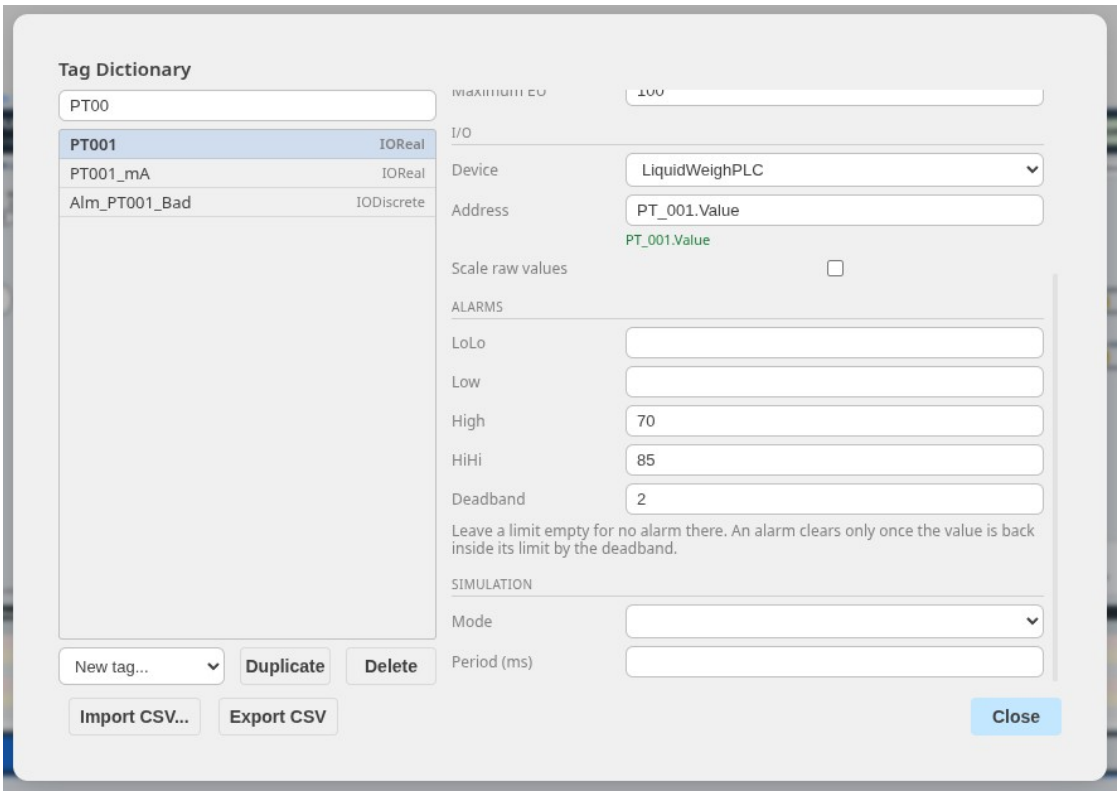



*Figure 7. Alarm limits and deadband for PT001*

Tags can also be exported to and imported from CSV (**Export CSV**, **Import CSV...**), which is the fastest way to create hundreds of tags from a PLC tag list: LiquidWeighHMI's 420 tags were entered that way.

## 5.3 Renaming, and finding and replacing tag references

Renaming a tag in the Tag Dictionary carries every reference in the project with it: the animation links on every page, the window and application scripts, QuickFunction bodies, and the recipe books. The dictionary says what it changed, and warns that what it changed outside the page you are on cannot be undone.

To point part of a screen at different tags -- a copied unit, say -- use **HMI > Find/Replace Tag References**. It lists every tag named by the current window, or by the objects selected, with how many times each is used:

- Type a replacement beside a tag, or use **Find** and **Replace with** at the top to change a piece of text in all of the names at once -- Pump1_ to Pump2_, for example. A tag left blank is not touched.
- **Validate** checks the replacements and reports anything they would break; **Apply** makes them all in one step, which one Undo reverses.
- Double-click a name to create or edit that tag without leaving the dialog, which is how a replacement that does not exist yet is made.

Expressions, scripts, color bands, trend pens and the tag names given to LinkIndirectTag and TrendSetPen are all updated. A dotfield moves with its tag, text inside comments and strings is left alone, and a QuickFunction's arguments are never mistaken for tags.

# 6 Screens and windows

Each page of the drawing is a window of the application, named by its page tab. LiquidWeighHMI has nine screens, the Header and Footer windows they share, and five popups.




*Figure 8. Page tabs: every page is a window*

*Table 9. Window types*

| Type | Behavior | Used in LiquidWeighHMI for |
|---|---|---|
| **Replace** | Closes any window it overlaps when it opens | The nine screens |
| **Overlay** | Opens on top and leaves other windows open | The header and the navigation bar |
| **Popup (modal)** | Stays on top; nothing beneath it can be touched until it closes | Faceplates and confirmations |

## 6.1 Window properties

Select a page and open **HMI > Window Properties**. The Device Faceplate is a popup with a title bar, 460 × 560 pixels at 730, 150 on the screen. The preview shows where it appears.

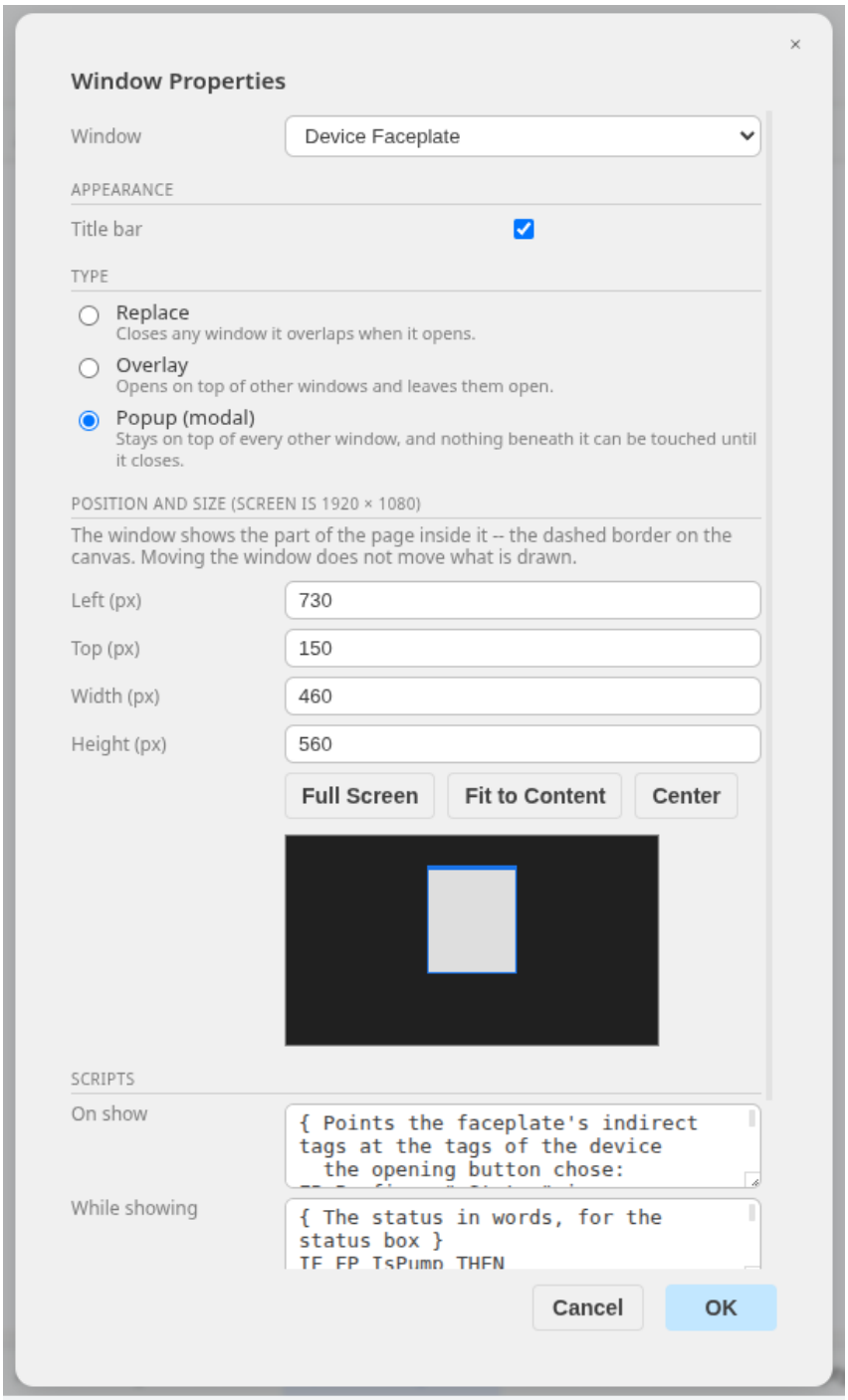



*Figure 9. Window Properties of the Device Faceplate popup*

A window shows the part of its page under its rectangle, so a popup's objects are drawn at the popup's position on its page. Figure 10 shows the Device Faceplate page in the editor. It serves every valve and pump (section 9.4), so a few objects for valves and for pumps share a place; visibility links show the right one while running.

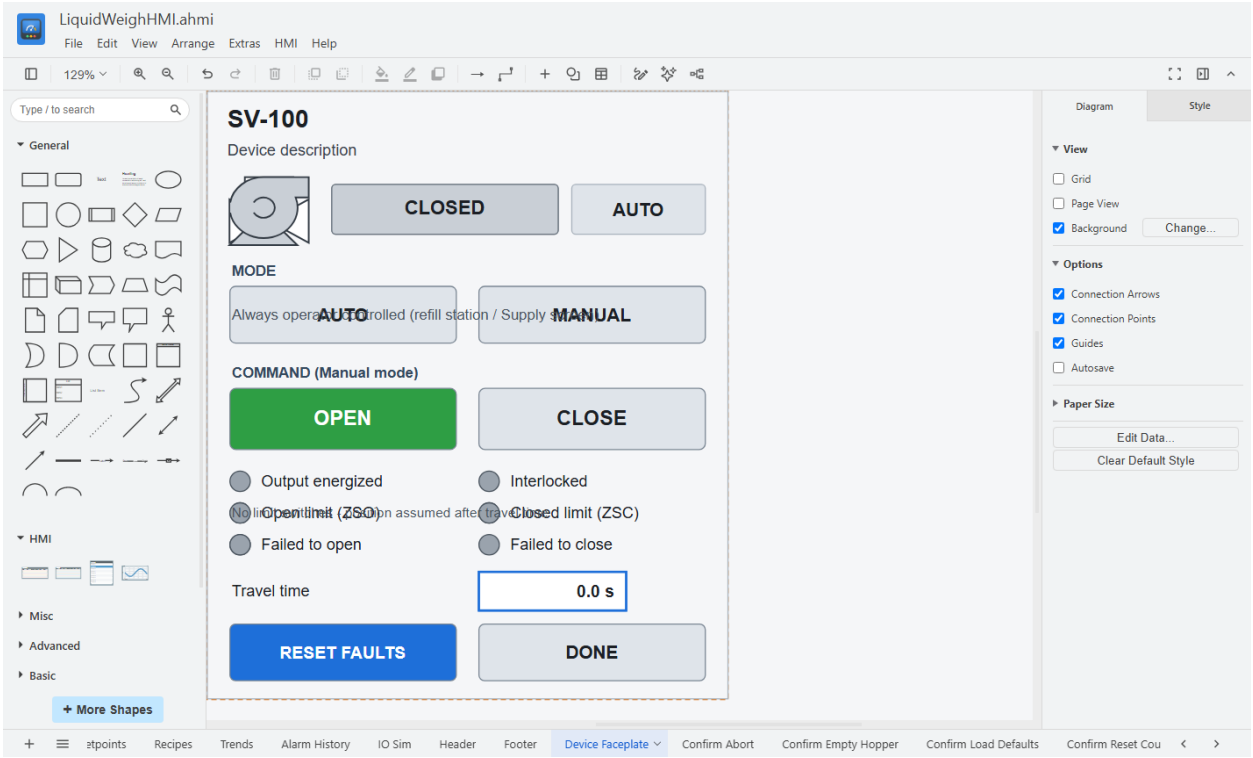



*Figure 10. The Device Faceplate page in the editor*

## 6.2 The header, the navigation bar and the screens

What every screen shares is drawn once, in two windows of its own. **Header** is an overlay 1920 × 70 at 0, 0 and **Footer** an overlay 1920 × 68 at 0, 1012; both open with the application and stay open. Each screen is a Replace window over the middle of the display, 1920 × 942 at 0, 70, so opening one closes only the screen before it: the header and the navigation bar are never overlapped, so they are neither closed nor drawn again, and a screen change rebuilds the middle alone.

The navigation bar in the Footer window is a row of buttons, each with a **Show Window** link to one screen. The screen being shown is named in the header and its button is highlighted, from two tags that each screen sets in its **On show** script:

```
ScreenTitle = "BATCH CONTROL"; ScreenNo = 2;
```

The header's title has a **Value Display** link on `ScreenTitle`, and each navigation button a **Fill Color / Discrete** link on its own number, such as `ScreenNo == 2` (chapter 8). The valve and pump symbols open the Device Faceplate from their Action Scripts instead, with the `ShowWindow` function, beside the symbol touched (section 9.5); its DONE button closes it with a **Hide Window** link.

# 7 Drawing objects

Screens are drawn with the editor's shapes: drag a shape from the Shapes panel onto the canvas, then set its fill, line and text in the Format panel. LiquidWeighHMI uses:

- Rectangles and rounded rectangles for panels, buttons and value boxes.
- Process symbols from **More Shapes > P&ID** (valves, pumps, vessels) and plain shapes for tanks and totes.
- Lines and connectors for piping.
- Text for labels and titles.

The **General** palette ends with two **arcs**: an open curve along a circle, **Arc (Circle)**, which stays round when resized, and one along an ellipse, **Arc (Ellipse)**, which fills its box. Like lines, arcs take an arrow or other end at either end, and their width can change from one end to the other. Select an arc and open the Style tab to set it:

*Table 10. Arc settings (Style tab, Arc section)*

| Setting | Meaning |
|---|---|
| **Start angle, End angle** | Where the arc begins and ends, in degrees measured as on a protractor: 0 at the right, increasing counterclockwise, so 90 is the top. The arc runs counterclockwise from the start angle to the end angle: 0 to 180 is the top half, and a start greater than the end, such as 315 to 225, sweeps through 0 |
| **Start line width, End line width** | The thickness of the line at each end; it changes evenly between them. The start width is the Line width of the Line section. A line whose widths differ is drawn solid, not dashed |
| **Start arrow, End arrow** | The end at each end: none, classic, block, open, oval, diamond, box and others, as on lines |
| **Start size, End size** | The size of each end |
| **Start filled, End filled** | Whether a closed end, such as a block or diamond, is filled or drawn as an outline |

Arcs are objects like any other: animation links such as Line Color, Blink, Visibility and Orientation work on them, for example to show a gauge sweep or a flow direction.

The **HMI** palette adds five objects: the **Alarm List**, the live list of active alarms at the bottom of every LiquidWeighHMI screen, the **Alarm History**, used on the Alarm History screen, the **Recipe List** (chapter 11), the **Trend** (chapter 12) and the **List Box** (chapter 14).




*Figure 11. The HMI palette: Alarm List, Alarm History, Recipe List, Trend and List Box*

**Note:** Keep objects inside the Screen frame, and group repeated symbols (a valve with its tag label) so they can be copied with their links.

To mirror an object, select it and choose **Arrange > Direction > Flip Horizontal** or **Flip Vertical**, or use the **Flip** buttons on the Arrange tab. A group is mirrored as a whole about its center: every object in it, including groups inside it, moves to its mirrored place and is flipped itself, and lines and rotations follow, so a pump symbol that faces right faces left. Text stays readable.

# 8 Animation links

Animation links make objects live. Select an object and open the **Animation** tab of the Format panel. The top of the tab lists every link type; click one to add it. Links already on the object are marked, and their settings appear below.




*Figure 12. The Animation tab for the START button, with its links marked*

*Table 11. Animation link families*

| Family | Links | What they do |
|---|---|---|
| **Touch links** | User Input, Pushbutton, Action Script, Show Window, Hide Window, Slider Horizontal and Vertical | Operator actions: enter a value, set or toggle a tag, run a script, open or close a window, drag a slider |
| **Line, Fill and Text Color** | Discrete, Analog, Discrete Alarm, Analog Alarm | Color from an on/off value, from value bands, or from a tag's alarm state |

| Family | Links | What they do |
|---|---|---|
| **Value / Movement** | Location, Width, Height, Percent Fill, Orientation | Move, size, fill or rotate an object in proportion to a value |
| **Miscellaneous** | Visibility, Blink, Enable | Show or hide, flash, and allow or block touch |
| **Value Display** | Value Display | Show a value or text in place of the object's label |

Numbers in links (ranges, limits) are expressions too, so they can come from tags. The sections below walk through the links LiquidWeighHMI uses.

> **Note:** Objects can be layered freely. A touch goes to the topmost object that has a touch link, and an object drawn over it with no touch link of its own — a label, a panel, a pipe, the unfilled outline over a Percent Fill shape — is seen through, with the pointing hand showing over it as well. A touch link on a group covers everything in the group. An object that does have a touch link keeps the touch even while its **Enable when** condition or an **Enable** link blocks it, so a button the operator may not use does nothing.

## 8.1 A valve: status color, fault blink and faceplate

Each valve symbol on the Process screen carries three links. SV-102 is selected in Figure 13.

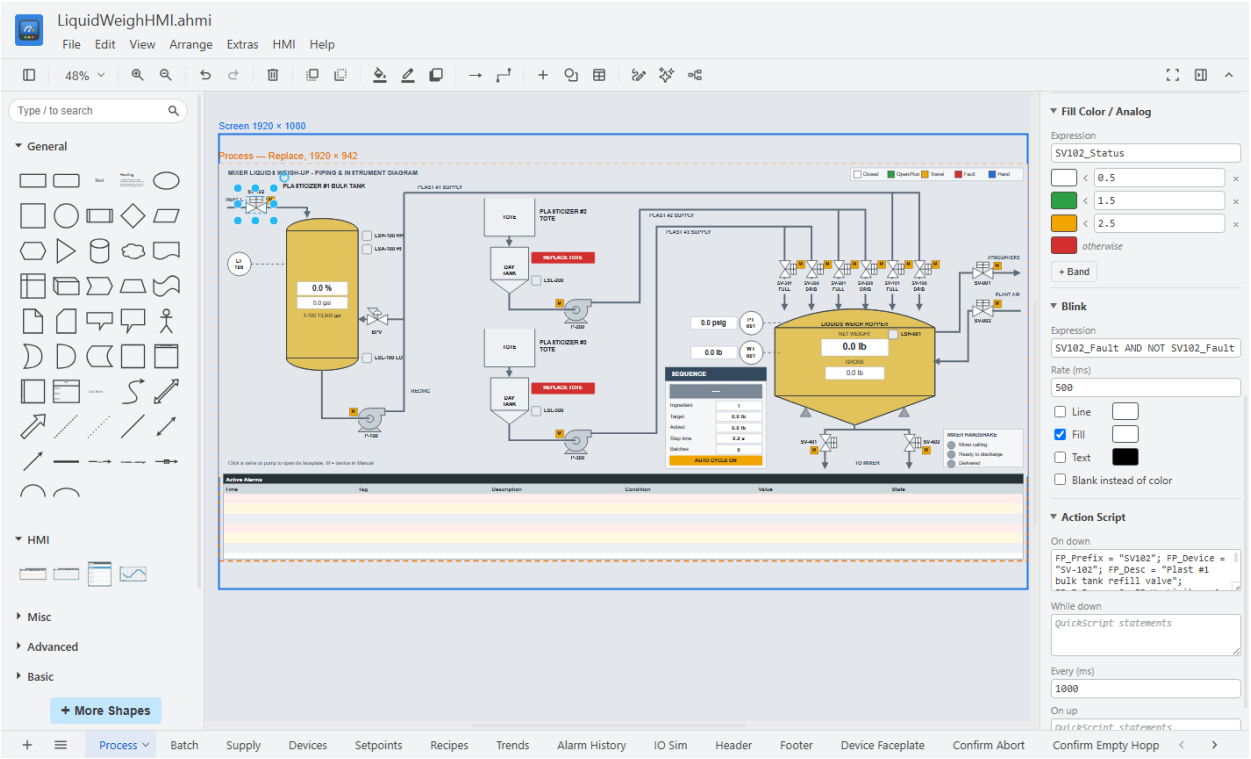



*Figure 13. The SV-102 valve and its links*

- **Fill Color / Analog** on SV102_Status: below 0.5 (closed) white, below 1.5 (open) green, below 2.5 (travel) amber, otherwise (fault) red. The first band whose limit the value is below wins.
- **Blink** on SV102_Fault AND NOT SV102_FaultAck: flashes the fill while a fault is unacknowledged.
- **Action Script** (On down): FP_Prefix = "SV102"; FP_Device = "SV-102"; ... tells the faceplate which device to show (section 9.4), and ShowWindow("Device Faceplate", 132, 80); opens it beside the valve (section 9.5).

## 8.2 The weigh hopper: percent fill

The hopper vessel fills with its net weight. **Percent Fill / Vertical** maps WT001 from 0 to 150 lb to 0 to 100 % fill, from the bottom up.

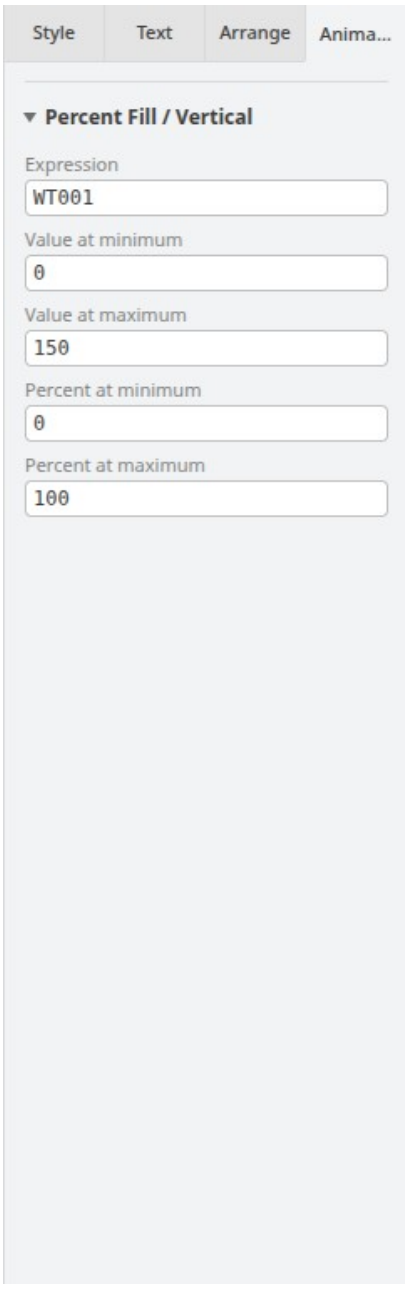



*Figure 14. Percent Fill on the weigh hopper*

**Note:** Percent Fill draws only the filled part of the object, outline included. Place an identical shape with no fill on top as the vessel outline, as LiquidWeighHMI does.

## 8.3 Hopper pressure: value display and alarm colors

The pressure box shows PT001 with one decimal and the suffix psig (**Value Display**), and its fill follows the tag's alarm state (**Fill Color / Analog Alarm**): normal white, High amber, HiHi red. The limits come from the tag in the Tag Dictionary, so they are set in one place.

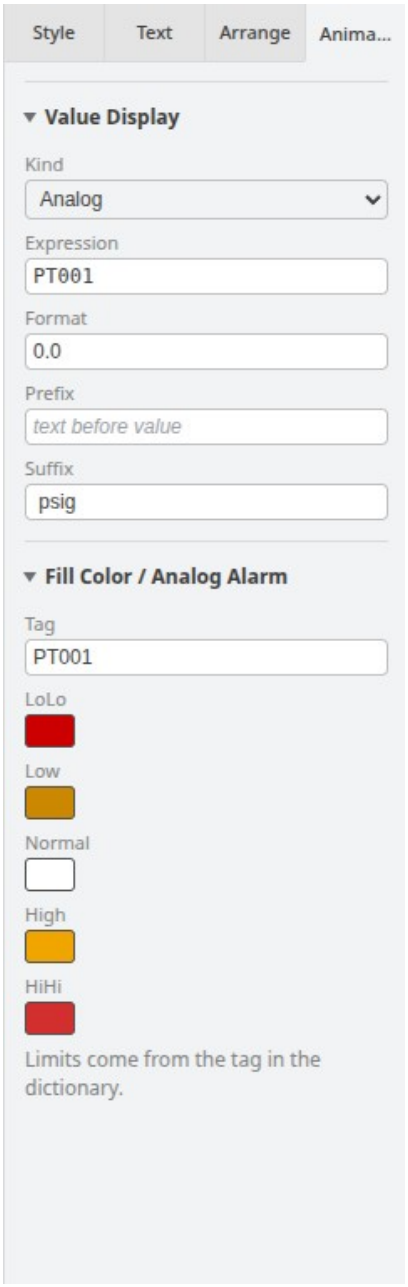



*Figure 15. Value Display and Fill Color / Analog Alarm on PT001*

## 8.4 Setpoints: user input

On the Batch screen the operator enters the batch volume by touching its value. **User Input** (analog) on `Rcp_TotalVol` accepts 0 to 12 gallons with the prompt "Total batch volume (gal, max 12)" and shows an on-screen keypad. A **Value Display** on the same object shows the current value.




*Figure 16. User Input and Value Display on the batch volume*

## 8.5 Commands: pushbutton and enable

The START button on the Batch screen is a **Pushbutton** that sets PB_Start when touched, allowed only while Seq_Idle AND Seq_StartPerm (Enable when). Its fill and text colors show when it is available. An **Enable** link on _AccessLevel > 0 blocks it until someone is logged in (chapter 10).

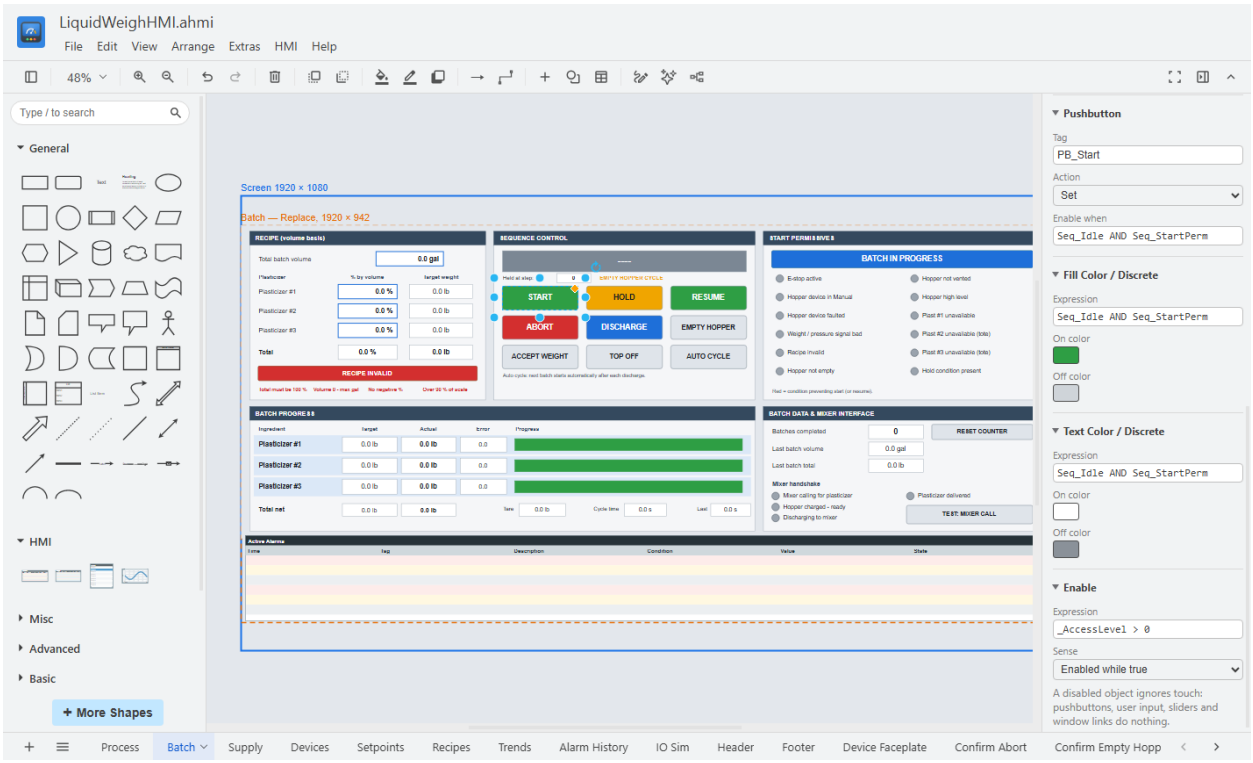



*Figure 17. The START button: Pushbutton, colors and Enable*

*Table 12. Pushbutton actions*

| Action | Effect |
|---|---|
| **Toggle** | Inverts the tag on each touch |
| **Set** | Writes 1 (START, HOLD, RESUME in LiquidWeighHMI) |
| **Reset** | Writes 0 |
| **Direct** | 1 while pressed, 0 when released (jog buttons) |

# 9 Expressions and scripts

Links use expressions, and scripts (Action Script links, window scripts and application scripts) use statements, both in the InTouch QuickScript style.

*Table 13. Expression syntax*

| Element | Examples |
|---|---|
| **Tags and dotfields** | PT001, SV001_Status, PT001.InAlarm, PT001.Acked, PT001.MaxEU |
| **Operators** | + - * / MOD, == <> < <= > >=, AND OR NOT, parentheses |
| **Literals** | 12.5, "IDLE" |
| **Functions** | Abs(x), Round(x), Text(value, "0.00") and the others in section 9.3 |
| **Statements (scripts)** | Pump_Run = 1;  IF cond THEN … ELSE … ENDIF; |
| **Local variables (scripts)** | DIM Count, Total AS Analog;  (section 9.7) |

*Table 14. System tags*

| Tag | Value |
|---|---|
| **_AlarmsActive** | Number of active alarms (the Alarms: n box in LiquidWeighHMI's header) |
| **_AlarmsUnacked** | Number of unacknowledged alarms |
| **_AckAll** | Write 1 to acknowledge every alarm |
| **_Username** | Logged-in user, or None |
| **_AccessLevel** | Logged-in user's access level, or 0 |
| **_DateString, _TimeString, _DateTime** | The date, the time and both as text: 2026-10-17, 14:05:09 and 2026-10-17 14:05:09.270. The last is what a date and time column takes as it stands |
| **_Year, _Month, _Day** | Numbers: the month 1 to 12 |
| **_Hour, _Minute, _Second, _Msec** | Numbers: the hour 0 to 23 |

The date and time tags are this PC's local time, worked out as they are read, so a script reading _Msec gets the millisecond it ran in; a box showing one is repainted once a second. All the system tags are read-only, apart from _AckAll.

## 9.1 Window and application scripts

A window can run a script when it opens (**On show**), repeatedly while it is open (**While showing**, every n ms) and when it closes (**On hide**). LiquidWeighHMI's **Header** window, open for as long as the application runs, turns the PLC's sequence step number into the text shown in the header, four times a second:

```
IF Seq_Step == 0 THEN StepText = "IDLE"; ENDIF;
IF Seq_Step == 5 THEN StepText = "TARE"; ENDIF;
IF Seq_Step == 11 THEN StepText = "PLAST #1 FAST FILL"; ENDIF;
...
```

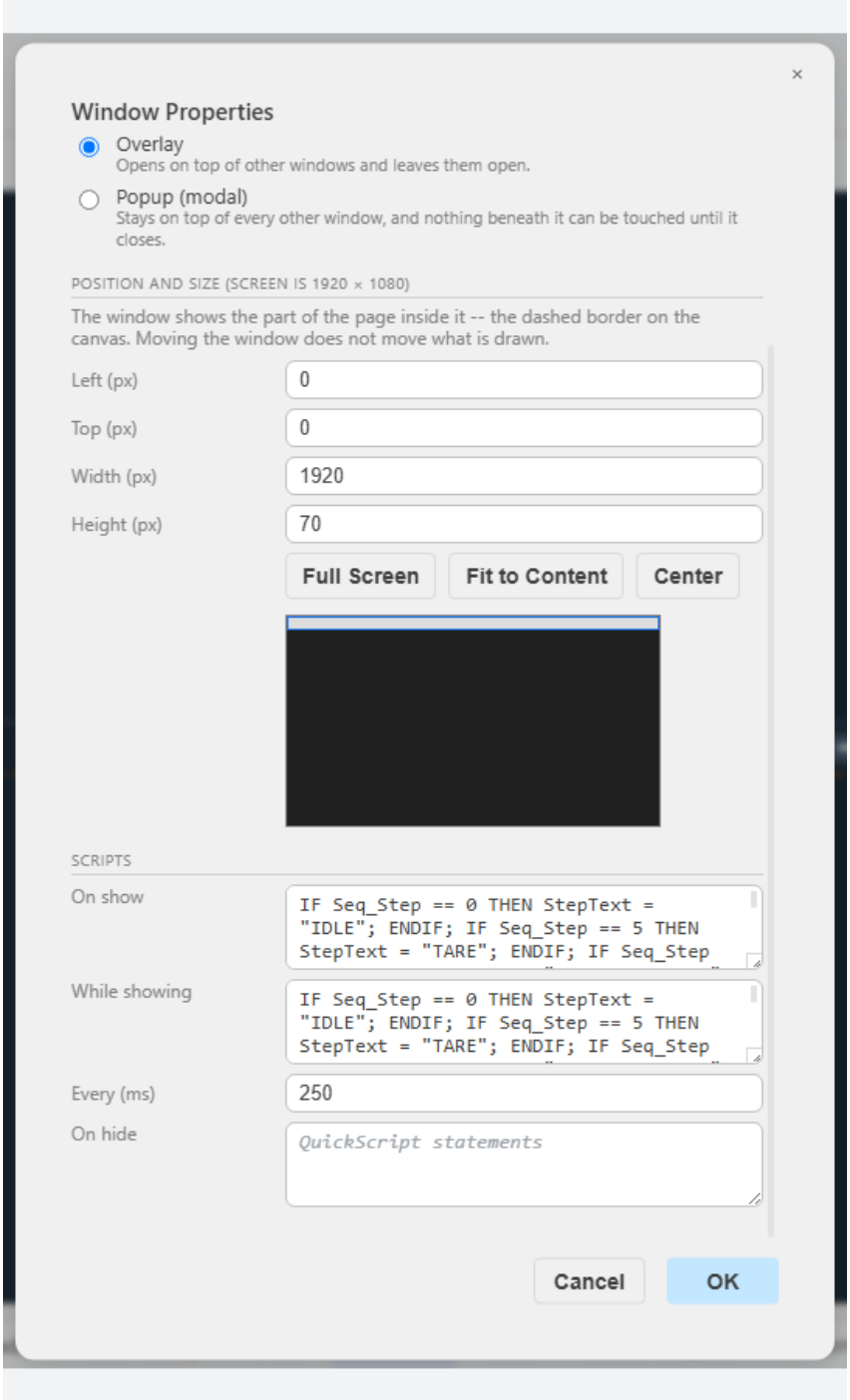



*Figure 18. The Header window: an overlay over the top of the screen, with its scripts*

A window's scripts run only while that window is open. Logic that must run all the time belongs in the **application scripts**, in the Scripts section of **HMI > Application Settings**:

- **On start** runs once when the application starts, before its first window opens, so it can set up what the windows show.

- **While running** runs every n ms (**Every (ms)**, 1000 unless set) for as long as the application runs, whichever windows are open.
- **On shutdown** runs once as the application stops, after its windows have run their On hide scripts. Retentive tags it writes are kept for the next start.

LiquidWeighHMISim, the copy of LiquidWeighHMI that needs no PLC (examples/LiquidWeighHMISim.ahmi), runs the whole controller program in its application scripts. **On start** does what the controller does on its first scan: it loads the setpoint defaults on the first run and sets the simulation's starting values. **While running** runs the 13 ladder routines, rung for rung, every 100 ms: the process simulation, sequence, interlocks, valves, pumps and alarms. Its tags are all memory tags, so START runs a whole batch on any PC.

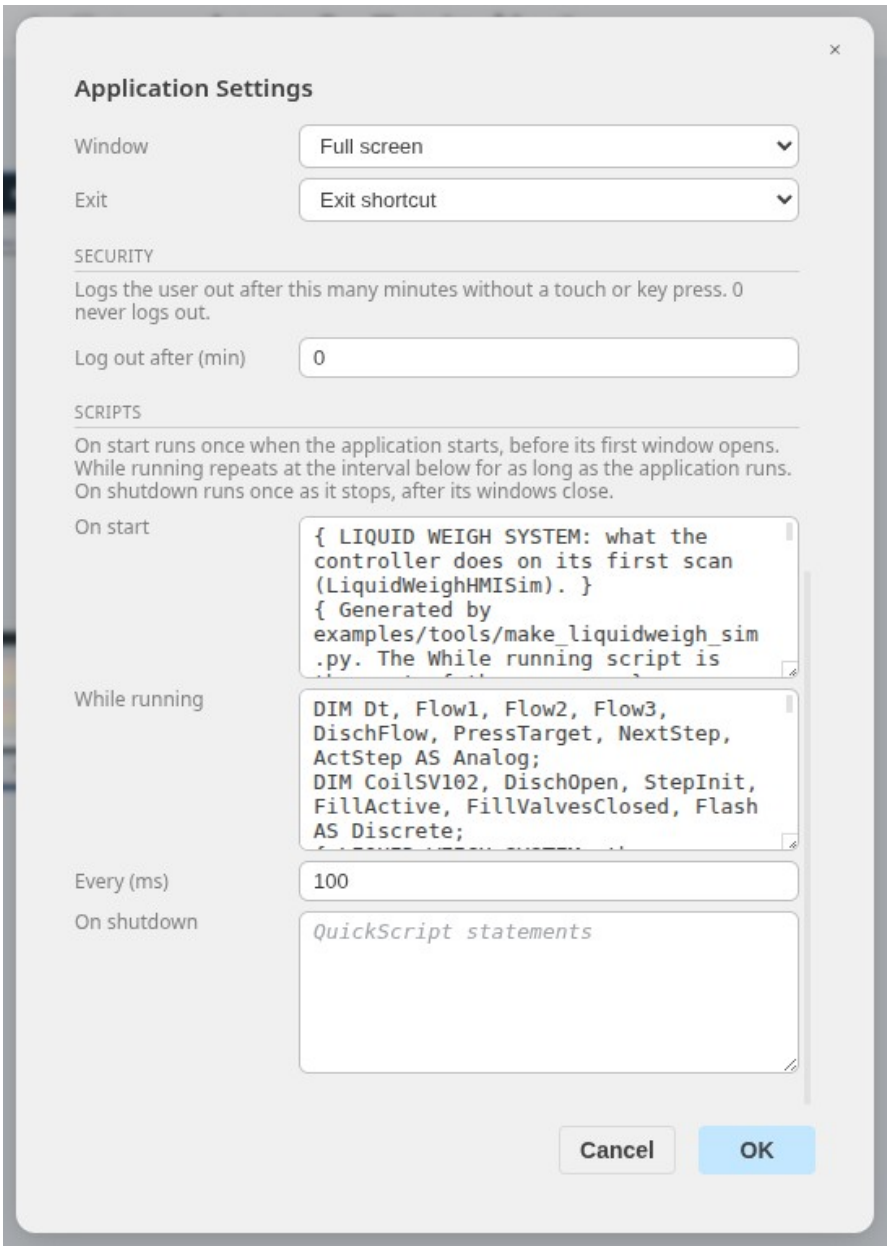



*Figure 19. LiquidWeighHMISim's application scripts: the controller program, every 100 ms*

- In a published application, On shutdown runs when the operator exits. When Windows shuts down or logs off, the application closes at once and On shutdown does not run.
- In Append HMI Web, each browser runs the application on its own, with its own application scripts and memory tags. On shutdown runs as the browser leaves the page, as far as the browser allows.

## 9.2 Action scripts

An **Action Script** link runs statements when the object is pressed (On down), repeatedly while held (While down) and when released (On up). The LOGIN button in LiquidWeighHMI's header logs in or out:

```
IF _Username == "None" THEN ShowLogin(); ELSE Logout(); ENDIF;
```




*Figure 20. The LOGIN button: Action Script, Value Display and colors*

## 9.3 Function reference

Functions take their arguments in parentheses, separated by commas. Any argument can be a literal (12.5, "Standard Blend"), a tag or a whole expression. Function names are not case-sensitive.

- **Math and text functions** return a value and can be used anywhere: in animation links and in scripts.
- **Security, indirect tag, window, screen, recipe and trend functions** do something (open a window, log in, link or write tags, save a file), so they can be called only from scripts: Action Scripts and window scripts. Validate reports one used in an animation link.
- Functions that succeed or fail return **1** or **0**, so a script can test them: IF Login(LoginName, LoginPassword) THEN ... ENDIF;.

*Table 15. Math and text functions (links and scripts)*

| Function | Arguments | What it does |
|---|---|---|
| **Abs(x)** | x: a number | The absolute value of x. Abs(-3.5) is 3.5 |
| **Sqrt(x)** | x: a number, 0 or more | The square root of x |
| **Sqr(x)** | x: a number | x squared (x × x) |
| **Int(x)** | x: a number | x with its fraction dropped. Int(7.9) is 7 and Int(-7.9) is -7 |
| **Round(x)** | x: a number | x rounded to the nearest whole number; a half rounds up. Round(2.5) is 3 |
| **Min(a, b)** | a, b: numbers | The smaller of a and b. Min(PT001, 50) is never more than 50 |
| **Max(a, b)** | a, b: numbers | The larger of a and b |
| **StringLen(text)** | text: text | The number of characters in text. StringLen(RecipeName) == 0 is true when no recipe is selected |
| **Text(value, format)** | value: a number format: text | The number as text, formatted by a picture such as "0.00". The digits after the point in format give the decimals, and the digits before it the minimum number of whole digits, padded with zeros. Text(3.14159, "0.00") is "3.14" and Text(7, "000") is "007" |

The text functions below work on message tags and literals. A character position counts from 1, and a count that runs past the end of the text stops there.

*Table 16. Text functions (links and scripts)*

| Function | Arguments | What it does |
|---|---|---|
| **StringLeft(text, count)** | text count: a number | The first count characters. StringLeft(_DateTime, 10) is the date |

| Function | Arguments | What it does |
|---|---|---|
| **StringRight(text, count)** | text, count | The last count characters |
| **StringMid(text, start, count)** | text, start, count | count characters from position start. StringMid(_DateTime, 12, 8) is the time |
| **StringTrim(text)** | text | text without the spaces at either end |
| **StringUpper(text), StringLower(text)** | text | text in capitals, or in small letters |
| **StringInString(text, find)** | text, find | The position where find starts in text, or 0 when it is not there |
| **StringReplace(text, find, with)** | text, find, with | text with every occurrence of find replaced by with |
| **StringFromIntg(number, base)** | number base: optional | The whole number as text, in base 2 to 36 (10 unless given). Base 16 gives small letters, so wrap it in StringUpper for FF |
| **StringFromReal(number, decimals, kind)** | number, decimals kind: optional | The number as text with that many decimals. kind "f" is a plain number (the default), "e" is exponential, and "g" is the shorter of the two with trailing zeros dropped, counting significant digits |
| **StringToIntg(text), StringToReal(text)** | text | The number the text starts with, or 0 when it holds none, so an operator's typing cannot break them |

*Table 17. Security functions (scripts only)*

| Function | Arguments | What it does |
|---|---|---|
| **ShowLogin()** | None | Opens the login window: user name, masked password and an on-screen keyboard. The window checks the password and logs the user in. Returns 1 |
| **Login(name, password)** | name, password: text | Logs in when the name and password match a user. Returns 1, or 0 when they do not. For custom login screens |
| **Logout()** | None | Logs the current user out: _Username becomes None and _AccessLevel 0. Returns 1 |
| **ChangePassword(old, new)** | old, new: text | Changes the logged-in user's password from old (the current password) to new (not empty), saved on this computer. Returns 1, or 0 when old is wrong or new is empty |
| **ShowUserManager()** | None | Opens the Users window while running, to add, change or remove users on this computer (section 10.2). Returns 1 |

*Table 18. Indirect tag function (scripts only)*

| Function | Arguments | What it does |
|---|---|---|
| **LinkIndirectTag(indirect, tag)** | indirect, tag: tag names as text | Points the indirect tag at the tag, replacing any earlier link; reading or writing the indirect tag then reads or writes that tag, in every window. The names can be built: LinkIndirectTag("FP_Status", "SV" + Text(Unit, "000") + "_Status"). Returns 1, or 0 with an Indirect Tag Error window when a name is unknown or the types do not match |

*Table 19. Window function (scripts only)*

| Function | Arguments | What it does |
|---|---|---|
| **ShowWindow(name, left, top, modal, wait)** | name left, top, modal, wait: optional | Opens the window (page) named by name (text), as a Show Window link does. left and top (screen pixels; "" keeps the window's own) move it for this showing; it still shows the same part of its page. modal (0 or 1) overrides a popup's modality: with 0 the windows beneath can still be touched. With wait 1 the script pauses until the window closes, so it can act on the operator's choice; by default it carries on at once. Like ShowRecipeSelect it must be a statement on its own or the whole right side of an assignment. Returns 1, or 0 with a Window Error window when the name is unknown, a position is not a number or the window would be off the screen |

*Table 20. Screen function (scripts only)*

| Function | Arguments | What it does |
|---|---|---|
| **ScreenToPDF()** | None | Saves the screen as the operator sees it, popups included, as a one-page PDF: landscape Letter, the screen fitted, centered and at least 1/4 inch from every edge. A Save window opens with the page's name as the file name (in Append HMI Web, the browser downloads the file). Returns 1 when the Save window opens; the script does not wait for it (section 9.6) |

In the recipe functions, book is the name of a recipe book defined in **HMI > Recipes** and name the name of a recipe in it: text of 1 to 64 characters. Names are not case-sensitive. Chapter 11 shows them at work.

*Table 21. Recipe functions (scripts only)*

| Function | Arguments | What it does |
|---|---|---|
| **RecipeSave(book, name)** | book, name | Saves the current values of the book's Save/Load tags as recipe name, replacing a recipe of that name. Fails if any of the values has bad quality |
| **RecipeLoad(book, name)** | book, name | Writes the recipe's values to the book's Save/Load tags |

| Function | Arguments | What it does |
|---|---|---|
| **RecipeDownload(book, name)** | book, name | Copies each Save/Load tag's value to its Upload/Download tag, usually sending the recipe to the PLC. name is not used: give "" or the selected recipe |
| **RecipeUpload(book, name)** | book, name | Copies each Upload/Download tag's value back to its Save/Load tag, usually bringing the PLC's values into the HMI. name is not used |
| **RecipeExport(book, name)** | book, name | Writes the recipe to a CSV file that the operator chooses (in Append HMI Web, the browser downloads it). Returns 1 when the file window opens |
| **RecipeImport(book, name)** | book, name | Reads a recipe CSV file that the operator chooses and saves it as recipe name. Returns 1 when the file window opens |
| **RecipeDelete(book, name, confirm)** | book, name confirm: optional | Deletes the recipe. With confirm 1, a Yes/No window asks the operator first; No returns 0 |
| **RecipeRename(book, name, newName)** | book, name, newName | Renames the recipe to newName, which must not already be in the book |
| **ShowRecipeSelect(book, x, y, w, h)** | book x, y, w, h: optional | Opens a window listing the book's recipes, with up and down arrow buttons, Select and Cancel, and returns the chosen name, or "" for Cancel. The script waits until the operator chooses, so it must be a statement on its own or the whole right side of an assignment: `Pick = ShowRecipeSelect("B");`. x, y, w, h place the window: position and size in screen pixels |

The recipe functions other than ShowRecipeSelect return 1 when they succeed and 0 when they fail. A failure also opens the Recipe Error window (section 11.5).

In the trend functions, `trend` is the name of a Trend object, set in its Animation tab, and `tag` the name of one of its pens, both as text: a literal in quotes or a message tag holding the name. Chapter 12 shows them at work.

***Table 22. Trend functions (scripts only)***

| Function | Arguments | What it does |
|---|---|---|
| **TrendSetPen(trend, tag)** | trend, tag | Selects the pen: the y axis shows its range, name and units, and its line is drawn 1 pixel thicker |
| **TrendPause(trend)** | trend | Freezes the plot. Recording goes on |
| **TrendPlay(trend)** | trend | Returns the plot to live, ending at now |
| **TrendScrollBack(trend)** | trend | Moves the start and end times back by half the time span, pausing a live trend first |

| Function | Arguments | What it does |
|---|---|---|
| **TrendScrollForward(trend)** | trend | Moves them forward by half the time span, stopping at now. The trend stays paused until TrendPlay |

Each trend function returns 1, or 0 with a Function Error window naming the call and the reason, such as a trend or pen that does not exist.

The **database functions** read and write SQL databases. There are 40 of them, and chapter 13 describes them all; these are the ones most applications use:

*Table 23. Database functions (scripts only)*

| Function | Arguments | What it does |
|---|---|---|
| **SQLConnect(ConnIdTag, source)** | ConnIdTag: an analog tag<br>source: text | Opens the data source named by source and writes the connection number into ConnIdTag |
| **SQLInsert(ConnId, table, bindList)** | ConnId<br>table, bindList: text | Writes one row from the current values of the bind list's tags |
| **SQLSelect(ConnId, table, bindList, where, orderBy)** | ConnId<br>the rest: text | Fetches the bind list's columns; where and orderBy may be "" |
| **SQLFirst(ConnId), SQLNext(ConnId)** | ConnId | Moves to the first or next row of the selection and writes it into the bind list's tags |
| **SQLErrorMsg(code)** | code: a result code | What a result code means, as text to show the operator |
| **SQLDisconnect(ConnId)** | ConnId | Closes the connection |

Each one returns 0 when it worked and a negative result code when it did not, except SQLNumRows, which returns the number of rows, and SQLErrorMsg, which returns text. A failure also opens a Database Error window.

The **list functions** fill and work a List Box object (chapter 14). In them, list is the name of a list box, set in its Animation tab, as text: a literal in quotes or a message tag holding the name. An item is text too, so a message tag or a literal, and an item is found whatever its case.

*Table 24. List functions (scripts only)*

| Function | Arguments | What it does |
|---|---|---|
| **ListClear(list)** | list | Removes every item and clears the selection |
| **ListAdd(list, item)** | list, item | Adds the item to the bottom of the list |
| **ListLoad(list, text)** | list, text | Adds the comma-separated, trimmed items in text to the bottom |

| Function | Arguments | What it does |
|---|---|---|
| **ListRemove(list, item)** | list, item | Removes the item; does nothing if the list has not got it |
| **ListSelect(list, item)** | list, item | Selects the item; does nothing if the list has not got it |
| **ListSort(list, order)** | list<br>order: "ASC", "A", "DESC" or "D", in any case | Sorts the items alphabetically, ignoring case, keeping the selection in view |

Each one returns 1, or 0 with a List Error window naming the call and the reason, such as a list that does not exist or a sort order it does not know.

## 9.4 Indirect tags: one faceplate for every device

LiquidWeighHMI has 11 valves and 3 pumps, and one **Device Faceplate** for all of them. Its objects use **indirect tags** (section 5): `FP_Status`, `FP_Manual`, `FP_CmdOn` and so on, which have no value of their own. When the faceplate opens, a script links each of them to the tag of the device being shown, so `FP_Status` reads `SV102_Status` for SV-102 and `P100_Status` for P-100. A change to the faceplate is made once and applies to every device.

Opening it takes two steps. First, the valve or pump symbol's Action Script names the device in memory tags and opens the faceplate beside the symbol (section 9.5):

```
FP_Prefix = "SV102"; FP_Device = "SV-102";
FP_Desc = "Plast #1 bulk tank refill valve";
FP_IsPump = 0; FP_HasLimits = 1; FP_OperatorOnly = 1;
ShowWindow("Device Faceplate", 132, 80);
```

Then the faceplate's **On show** script builds each tag name from the prefix and links the indirect tags. A pump links the shared command tags to its Start and Stop, a valve to Open and Close:

```
LinkIndirectTag("FP_Status", FP_Prefix + "_Status");
LinkIndirectTag("FP_Manual", FP_Prefix + "_Manual");
...
IF FP_IsPump THEN
   LinkIndirectTag("FP_CmdOn", FP_Prefix + "_CmdStart");
   LinkIndirectTag("FP_CmdOff", FP_Prefix + "_CmdStop");
   LinkIndirectTag("FP_Time", FP_Prefix + "_FailTime");
ELSE
   LinkIndirectTag("FP_CmdOn", FP_Prefix + "_CmdOpen");
   LinkIndirectTag("FP_CmdOff", FP_Prefix + "_CmdClose");
   LinkIndirectTag("FP_Time", FP_Prefix + "_TravelTime");
   IF FP_HasLimits THEN
      LinkIndirectTag("FP_ZSO", FP_Prefix + "_ZSO");
```

```
        LinkIndirectTag("FP_ZSC", FP_Prefix + "_ZSC");
    ENDIF;
ENDIF;
```

- **Heading:** Value Displays of FP_Device and FP_Desc.
- **Valve or pump:** visibility links on FP_IsPump choose the symbol and status colors. Texts that differ, such as OPEN or START, are Discrete Value Displays of FP_IsPump.
- **Variants:** FP_HasLimits shows the limit-switch lamps or a note, and FP_OperatorOnly replaces the mode buttons of the refill valve with a note.
- **Status:** a While showing script turns FP_Status into the words shown (FP_StateText).
- **Dotfields:** FP_Status.Name would show SV102_Status; every dotfield describes the linked tag.

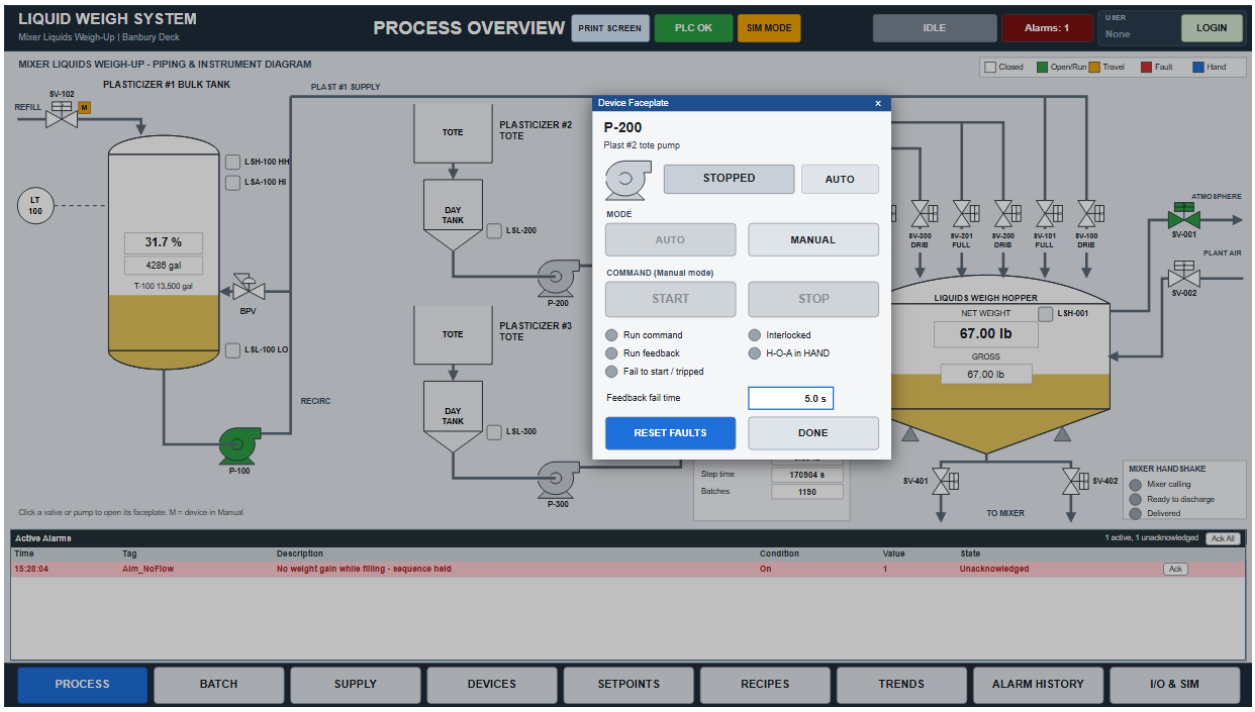



*Figure 21. The same faceplate showing pump P-200*

**Note:** Writing to an indirect tag writes the linked tag: the faceplate's MANUAL button sets FP_CmdManual, which is SV100_CmdManual while SV-100 is shown. Before the first link an indirect tag has bad quality and writes to it are ignored.

## 9.5 Opening windows from scripts: ShowWindow

A Show Window link always opens its window where the window is defined. The `ShowWindow` function (section 9.3) does the same from a script, and can also move the window, make a popup modal or not, and pause the script until the window closes. LiquidWeighHMI uses it in two ways.

## Beside the device touched

Every valve and pump symbol ends its Action Script with a ShowWindow call that places the Device Faceplate beside the symbol, so the operator still sees the device being operated. The faceplate still shows its own part of its page; only its place on the screen changes. P-100, at the bottom left of the Process screen, opens it at 414, 398:

```
FP_Prefix = "P100"; FP_Device = "P-100"; ...
ShowWindow("Device Faceplate", 414, 398);
```

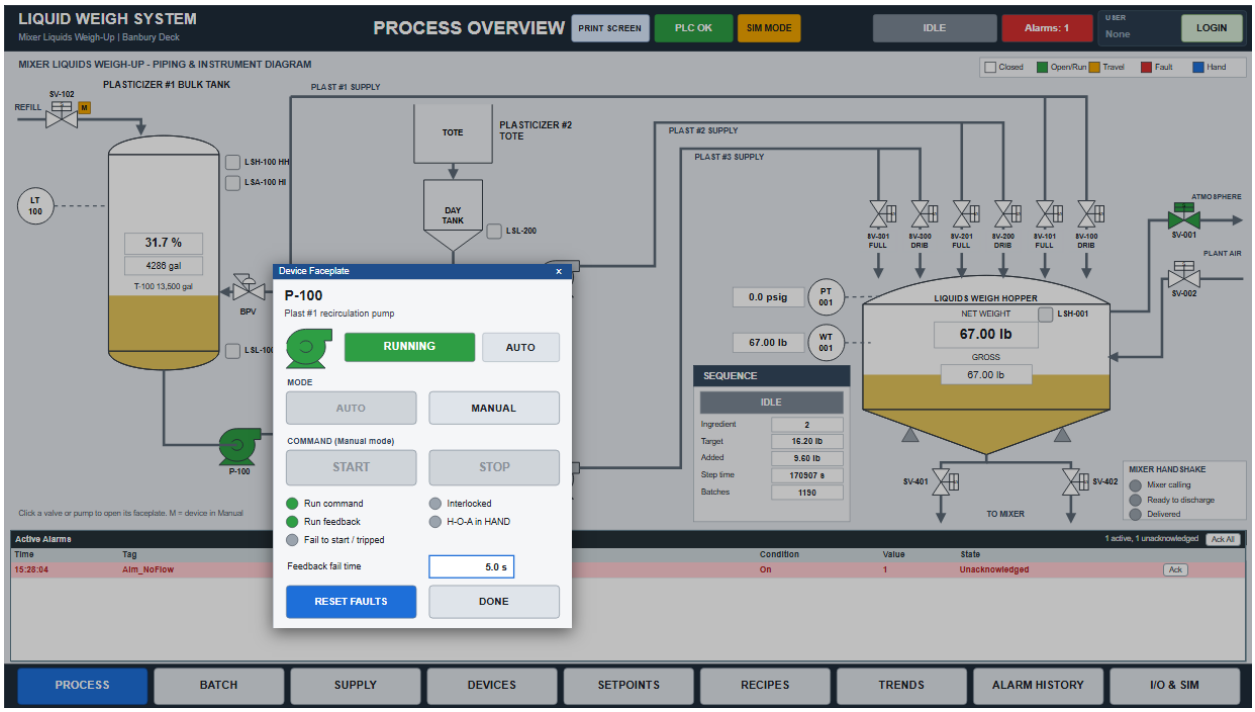



*Figure 22. The Device Faceplate opened beside pump P-100*

## Waiting for an answer

ABORT on the Batch screen asks first. Its Action Script opens the Confirm Abort popup with wait (the fifth argument), so the script pauses at ShowWindow until the popup closes, then acts on the answer:

```
IF Seq_Running OR Seq_Held THEN
  AbortConfirmed = 0;
  ShowWindow("Confirm Abort", "", "", 1, 1);
  IF AbortConfirmed THEN
    PB_Abort = 1;
  ENDIF;
ENDIF;
```

The popup's YES button sets AbortConfirmed = 1 and closes it; NO, or the close box, just closes it. "" keeps the popup where it is defined, and 1 keeps it modal.

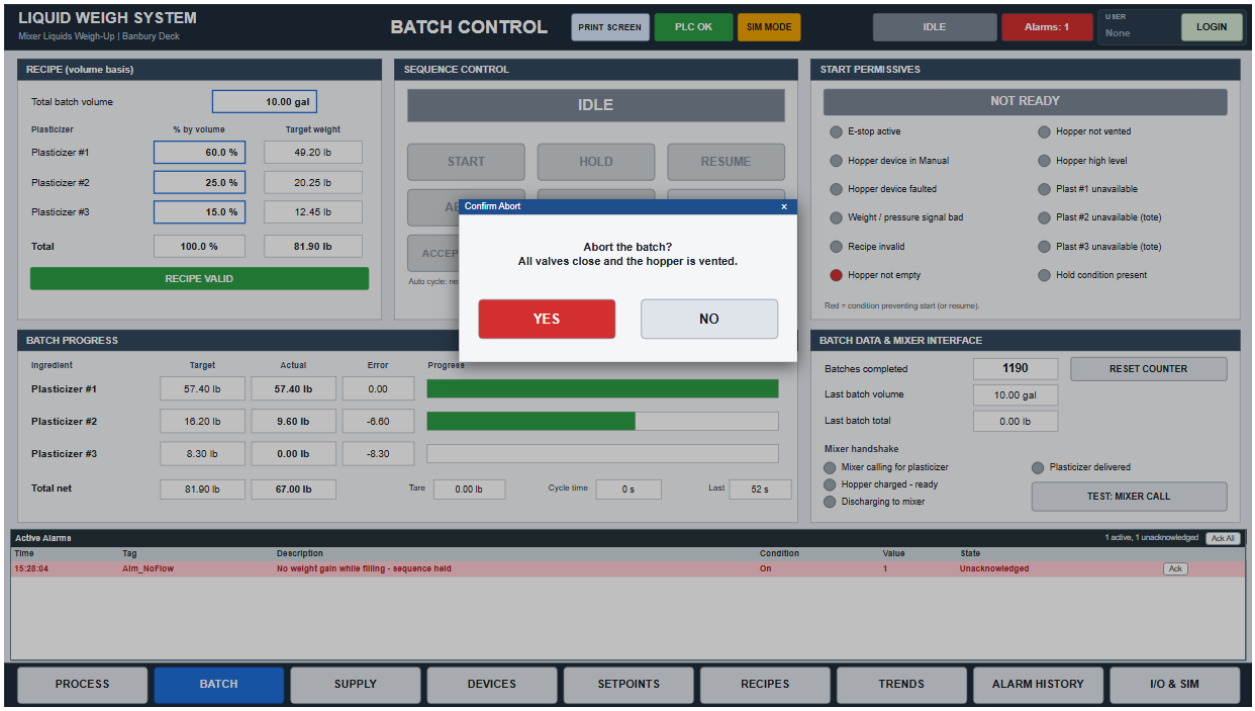



*Figure 23. Confirm Abort, opened by ShowWindow with wait: the ABORT script waits for the answer*

- While a script waits, the rest of the application runs normally; only that script is paused.
- A waiting ShowWindow must be a statement on its own (or the whole value of an assignment), as here, inside an IF if needed.
- If the window running the script closes, or the application stops, before the answer, the rest of the script does not run.

## 9.6 Saving the screen as a PDF: ScreenToPDF

ScreenToPDF() keeps a record of a screen: a shift report, a batch at a given moment, or a fault to send to maintenance. LiquidWeighHMI's header has a **PRINT SCREEN** button, whose Action Script is one line:

```
ScreenToPDF();
```

The function saves what the operator sees: the screen with its live values, and any popup or overlay window open on it. The PDF has one landscape Letter page. The screen is fitted to the page as large as it goes, centered, with at least 1/4 inch of margin on every side, and is drawn at the screen's own resolution even when it is shown smaller.

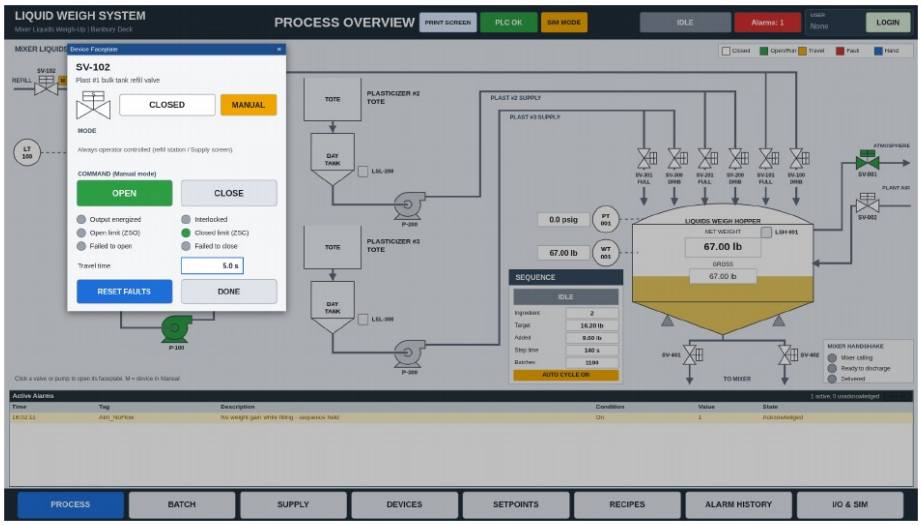



*Figure 24. The PDF saved from the Process screen with the Device Faceplate open*

- A Save window asks where to save the file. The file name offered is the name of the page shown as the screen, such as Process.pdf. Characters that a file name cannot hold (\ / : * ? " < > |) become _, so a page named "Tank 1/2" is offered as Tank 1_2.pdf.
- Like RecipeExport, the function returns 1 as soon as the Save window opens, and the rest of the script runs at once; it does not wait for the operator. A failure, such as a folder that cannot be written, opens the Screen to PDF Error window. Cancel saves nothing and shows no error.
- In Append HMI Web the browser downloads the file, to its downloads folder or where it asks, depending on the browser's settings.
- Pictures on the screen print when they are embedded in the project, as pictures added in Append HMI Studio are. In the desktop apps, a screen that shows a picture by its web address is captured from the window instead, at the size it is shown.

## 9.7 Local variables: DIM

A script can keep working values in **local variables** instead of tags. DIM declares them, with their type:

```
DIM Total, Average AS Analog;
DIM Found AS Discrete;
DIM Label AS Message;
Total = Bat_Actual0 + Bat_Actual1 + Bat_Actual2;
Average = Total / 3;
Found = Average > 10;
Label = "Batch " + Text(Bat_Count, "0");
IF Found THEN StepText = Label; ENDIF;
```

*Table 25. Local variable types*

| Type | Holds | Starts at |
|---|---|---|
| **Analog** | A number. Text that is a number, such as "12", becomes that number | 0 |
| **Discrete** | 1 or 0. Anything true becomes 1 | 0 |
| **Message** | Text. A number becomes its text | "" |

- **One script, one run:** a local variable belongs to the script that declares it and lasts while that script runs. Each run starts it again at 0 or "". A script paused at ShowWindow with wait, or at ShowRecipeSelect, keeps its values until it carries on. Values that must last from one run to the next belong in memory tags.
- **Declare first:** DIM goes at the top level of the script, not inside IF, before the names are used. Several DIM statements can declare several types.
- **Strict names:** a local variable cannot have the name of a tag, whatever its case, or of a system tag, and a name can be declared only once. Validate reports a clash, including one with a tag added later. Names are not case-sensitive, and local variables have no dotfields.
- **Not tags:** a local variable is not sent to a device or to other windows, and a write to it costs nothing. LiquidWeighHMISim keeps everything the controller works out afresh on each scan in local variables: the time since the last pass, flows, the next sequence step and the flasher.

## 9.8 QuickFunctions

A **QuickFunction** is script logic with a name, kept in the project and called like any built-in function. Write a rule once and use it everywhere instead of copying an expression into every object. Open **HMI > QuickFunctions** to define them.

Each one has a name, up to eight typed arguments, and a body. The body reads the arguments like variables and gives a value back with Return:

```
{ Scaled(Raw, Span): the raw count as a percentage of Span }
IF Span == 0 THEN
    Return 0;
ENDIF;
Return Raw * 100 / Span;
```

Call it from a script, Tank_Percent = Scaled(Tank_Raw, 4000);, or straight from an animation link, Scaled(Tank_Raw, 4000) in a Value Display or a percent fill.

*Table 26. What a QuickFunction may do*

| Rule | Why |
|---|---|
| **A function that writes a tag or calls an action function can be used in scripts only** | An animation link runs on every repaint, and acting on every repaint is not what an animation is for. The dialog says which of the two each function is, and Validate reports a link that calls one |
| **Return gives the value back and stops there, wherever it is** | Including inside IF. Return on its own stops with the default. A function that never returns gives 0, and Validate says so |
| **The value carries its quality** | A function returning a tag from a dead device returns bad quality, and the display shows #### |
| **ShowRecipeSelect and ShowWindow with wait cannot be used in a body** | They pause a script until the operator answers, and a function cannot be paused. Call them from the script that calls the function. A plain ShowWindow, which opens a window and carries on, is allowed |
| **A function may call other QuickFunctions, but not itself** | Directly or through a chain. Validate reports the whole cycle |

Three buttons under the body help while it is being written: **Validate** checks the script on demand and reports what it finds, **New Tag** makes a tag the script needs without leaving it, and **Browse Tags** lists the tags with a filter and copies the one picked to the clipboard, to paste into the body.

- **Names are strict:** a function or an argument cannot take the name of a tag, a system tag, a built-in function or a keyword, for the same reason a local variable cannot -- inside the body a name has to mean one thing. Return is a keyword only where a statement begins, so a project with a tag called Return keeps working.

- **Arguments are typed** Analog, Discrete or Message, and what is passed is converted to that type, exactly as a local variable is.
- **Cost:** a function used in an animation link runs every time that object repaints. Keep the body short, and leave heavy work to a window or application script that writes a tag the link reads.
- **Renaming a tag** reaches into the bodies, and leaves arguments of the same name alone.

# 10 Alarms and security

## 10.1 Alarms

Alarms come from the limits set on tags (chapter 5). While the application runs, every alarmed tag is watched, whether or not a screen shows it:

- Entering alarm, or escalating (High to HiHi), raises an alarm that needs acknowledging.
- An alarm stays in the Alarm List until it has returned to normal **and** been acknowledged (ISA-18.2).
- The Alarm List object has an **Ack All** button and an **Ack** button on each row. A script can acknowledge with `_AckAll = 1;` or `Tag.Acked = 1;`.
- Every alarm event is written to a daily history file kept for 90 days; the Alarm History object shows it (Figure 51).
- Both objects take their title, font size, columns and colors from the **Animation** tab. **Show title bar** turns the heading off for a table that sits under a title of your own; the Alarm List's counts and its Ack All button are in that bar and go with it, so acknowledge from a button that sets `_AckAll`.

The **Colors** rows give each part of the table a background and a text color, in that order (Figure 25). An Alarm List colors its title bar, its column header and its three row states; an Alarm History colors its title bar, its column header and its four events. **Default colors** puts every one of them back.

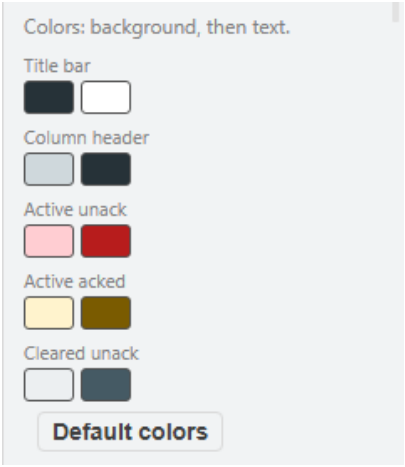



*Figure 25. The colors of an Alarm List, in the Animation tab*

*Table 27. What an alarm object colors*

| Alarm List | Alarm History | Shown when |
|---|---|---|
| Title bar | Title bar | The heading, while Show title bar is ticked |
| Column header | Column header | The row of column names |
| Active unack | | The alarm is active and not yet acknowledged |
| Active acked | | The alarm is active and has been acknowledged |

| Alarm List | Alarm History | Shown when |
|---|---|---|
| Cleared unack | | The alarm has returned to normal but is not acknowledged |
| | Alarm | The tag entered alarm, or escalated |
| | Return | The tag returned to normal |
| | Ack | An operator or a script acknowledged it |
| | Change | The alarm eased back to a lower limit |

- An Alarm List has three row states, not four: an alarm leaves the list once it has both returned to normal and been acknowledged, so there is no cleared and acknowledged row to color.
- A color you set covers every priority. Until you set one, an unacknowledged HiHi or LoLo row keeps the stronger shade that marks it out from a Hi or Lo one.
- The frame is the object's own, from the **Style** and **Text** tabs: **Fill** is the background behind the rows, **Line** the border and its width, and the font family, size and color the text.

## 10.2 Users and access levels

Open **HMI > Users** to define who can log in. Each user has a name, a password (stored only as a salted hash) and an access level from 0 to 9999.

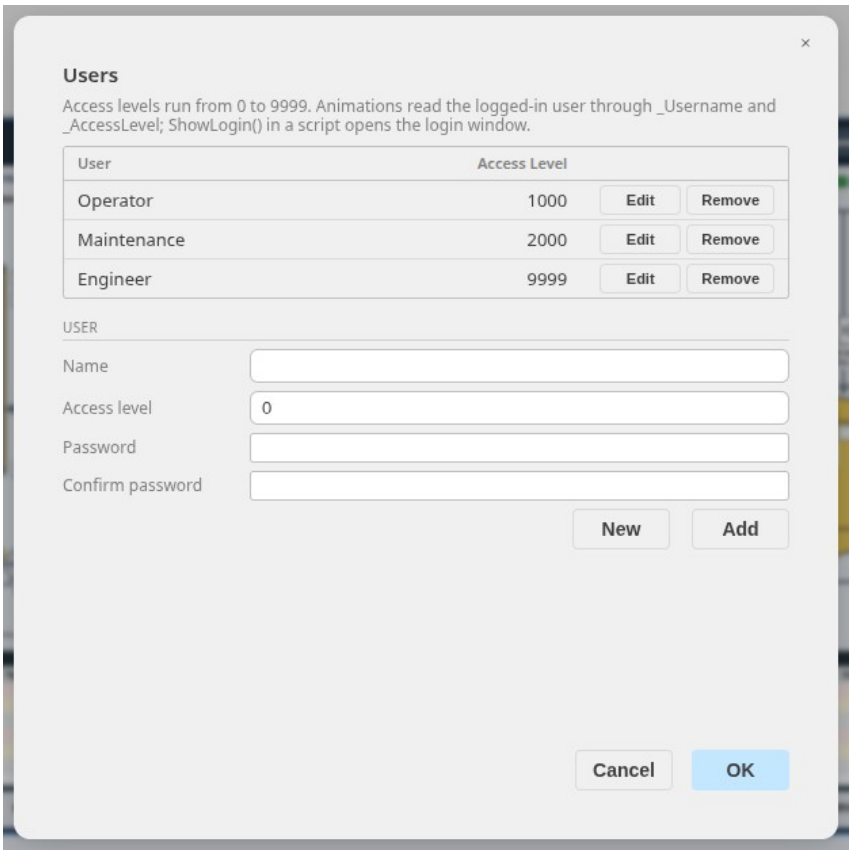



*Figure 26. The Users dialog with LiquidWeighHMI's three users*

*Table 28. LiquidWeighHMI users*

| User | Access level | Can |
|---|---|---|
| **Operator** | 1000 | Run batches |
| **Maintenance** | 2000 | Also operate devices in manual |
| **Engineer** | 9999 | Everything, including setpoints and simulation |

Screens use `_AccessLevel` to protect controls: an **Enable** link such as `_AccessLevel >= 2000` blocks touch on an object, and a **Visibility** link can hide it. The script function `ShowLogin()` opens the built-in login window (Figure 27); `Login(name, password)`, `Logout()`, `ChangePassword(old, new)` and `ShowUserManager()` support custom login screens.

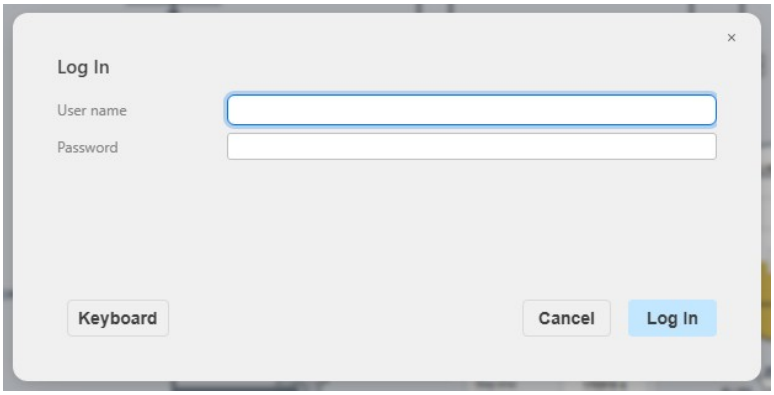



*Figure 27. The login window opened by ShowLogin()*

# 11 Recipes

A **recipe** is a named set of tag values that an operator saves and later loads again: the settings of a product, a grade or a batch size. Recipes are kept in **recipe books**. A book lists the tags its recipes hold:

- **Save/Load tags**: the values a recipe stores, usually memory tags that the operator edits on screen. `RecipeSave` reads them and `RecipeLoad` writes them.
- **Upload/Download tags** (optional): a second tag paired with each Save/Load tag, usually the PLC tag the value is meant for. `RecipeDownload` copies the Save/Load values to them and `RecipeUpload` copies them back.

Keeping the edited values in memory tags means the operator can prepare a recipe without disturbing the PLC, and send it with one Download when the process is ready.

## 11.1 Recipe books

Open **HMI > Recipes** to define the books. LiquidWeighHMI has one, **Plasticizer Blends**, that holds the Batch screen's recipe: the total batch volume and the share of each of the three plasticizers.

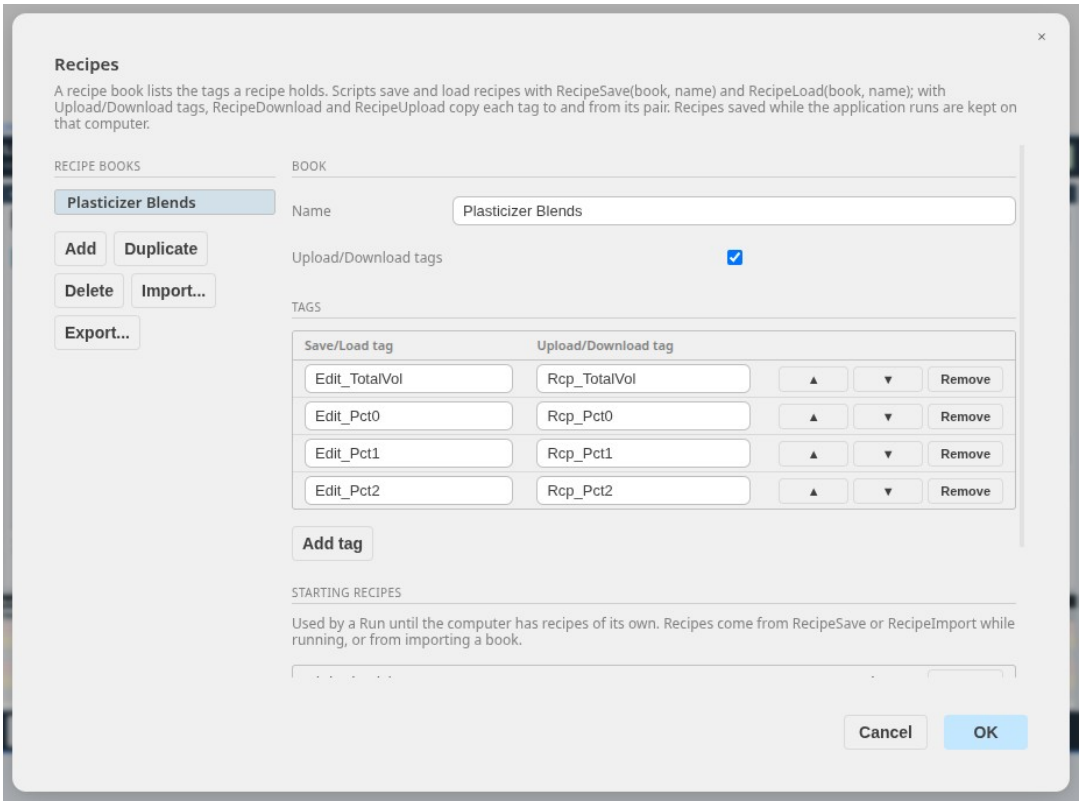



*Figure 28. HMI > Recipes: the Plasticizer Blends book and its tags*

- **Add**, **Duplicate** and **Delete** manage the books; **Name** renames the selected one.
- **Upload/Download tags** adds the second column of tags.
- **Add tag** adds a row. The arrow buttons reorder the rows and **Remove** deletes one. A tag that is not in the Tag Dictionary is marked.
- **Import…** and **Export…** read and write books, with their starting recipes, as a JSON file, to copy them between projects.
- **OK** checks every book and saves them into the project. Validate also reports books whose tags no longer exist.

*Table 29. The tags of the Plasticizer Blends book*

| Save/Load tag | Upload/Download tag | Holds |
|---|---|---|
| **Edit_TotalVol** | Rcp_TotalVol | Total batch volume, gal (PLC tag Recipe.TotalVol_gal) |
| **Edit_Pct0** | Rcp_Pct0 | Plasticizer #1, % by volume (Recipe.Pct[0]) |
| **Edit_Pct1** | Rcp_Pct1 | Plasticizer #2, % by volume (Recipe.Pct[1]) |
| **Edit_Pct2** | Rcp_Pct2 | Plasticizer #3, % by volume (Recipe.Pct[2]) |

The Edit_ tags are retentive memory tags, so the recipe being edited survives a restart. The PLC works out each plasticizer's target weight from the volume, the percentages and the densities on the Setpoints screen.

A book can also carry **starting recipes**: recipes stored in the project, which a computer uses until it saves recipes of its own. LiquidWeighHMI ships with five.

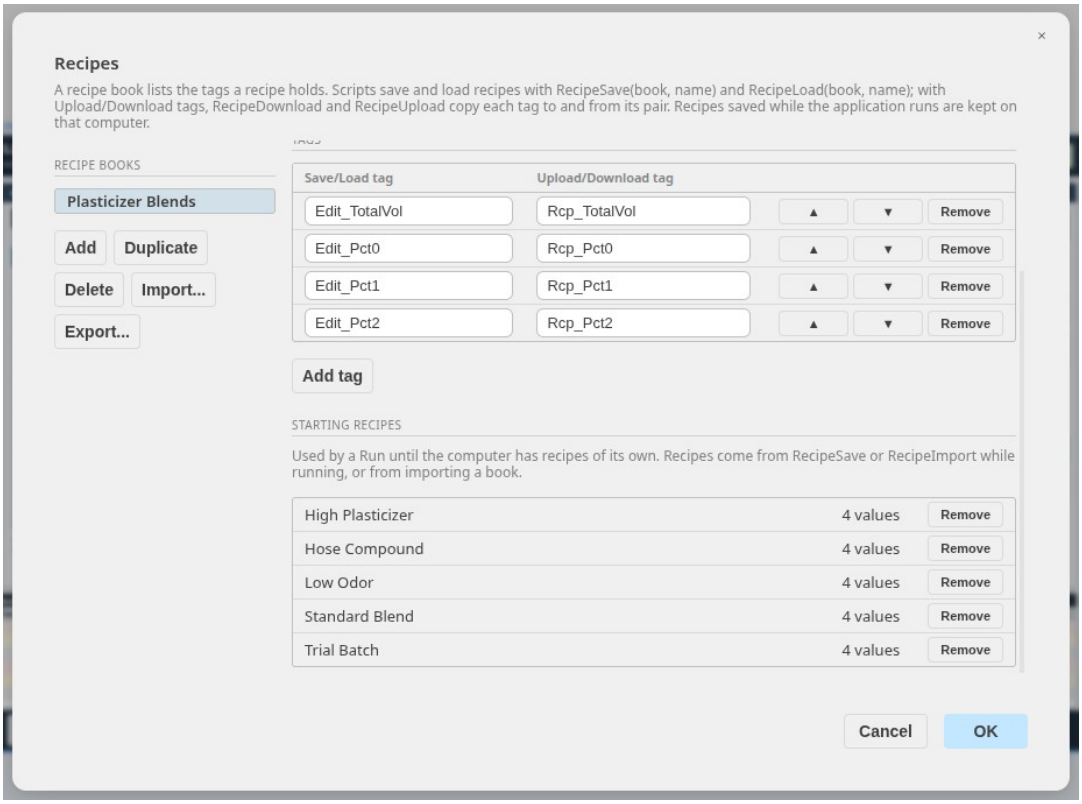



*Figure 29. The starting recipes of Plasticizer Blends, below its tags*

*Table 30. LiquidWeighHMI's starting recipes*

| Recipe | Volume (gal) | Plasticizer #1 (%) | Plasticizer #2 (%) | Plasticizer #3 (%) |
|---|---|---|---|---|
| **Standard Blend** | 10 | 60 | 25 | 15 |
| **High Plasticizer** | 10 | 70 | 20 | 10 |
| **Hose Compound** | 9 | 45 | 35 | 20 |
| **Low Odor** | 8 | 50 | 50 | 0 |
| **Trial Batch** | 4 | 40 | 30 | 30 |

## 11.2 The Recipes screen

LiquidWeighHMI's **Recipes** screen, on the RECIPES button of the navigation bar, puts the recipe functions to work. It is built only from standard objects, links and scripts, so it can be copied into other applications.

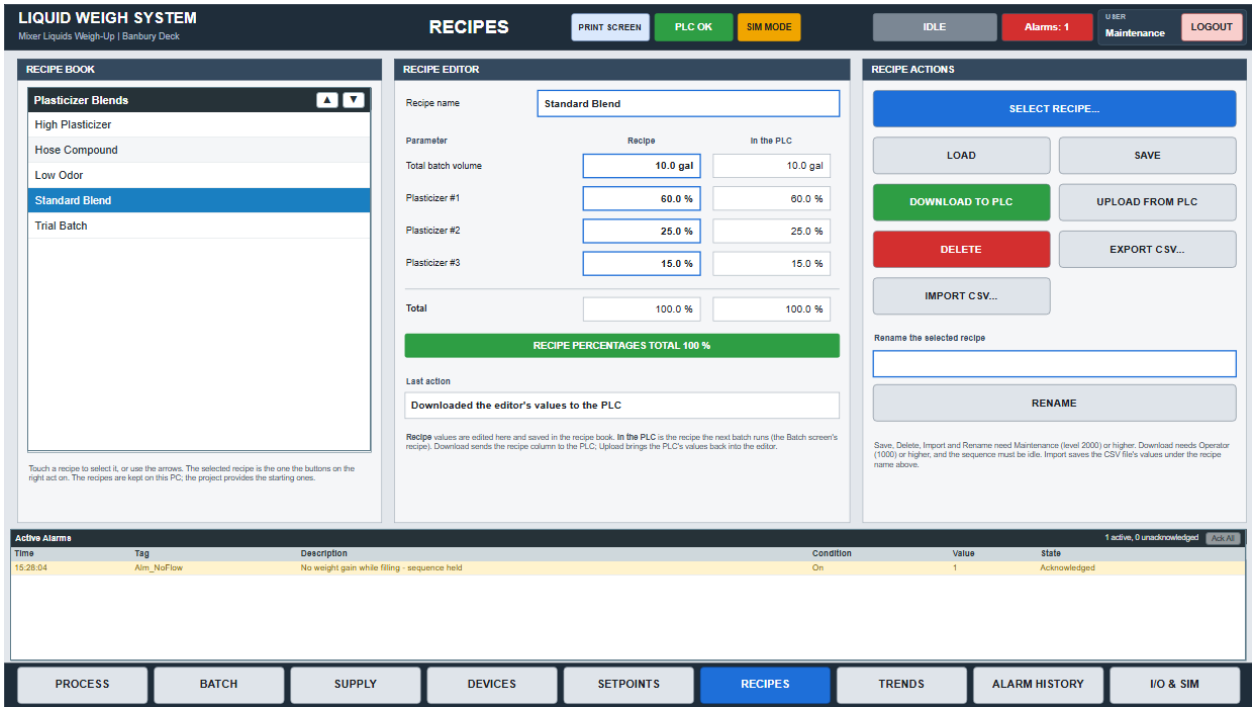



*Figure 30. The Recipes screen at Run: Standard Blend selected, loaded and downloaded to the PLC*

- **Recipe book** (left): a Recipe List object showing the book's recipes. Touching one, or the arrow buttons in its heading, selects it by writing its name to the message tag RecipeName.
- **Recipe editor** (center): the recipe name, and for each value the editable Save/Load tag (Recipe column) beside its PLC tag (In the PLC column). The total and the green or red banner check that the percentages add up to 100 %. **Last action** shows the result of the last button.
- **Recipe actions** (right): one button for each recipe function, acting on the recipe named in RecipeName.

*Table 31. The Recipes screen's buttons*

| Button | Calls | Allowed for |
|---|---|---|
| **SELECT RECIPE...** | ShowRecipeSelect, then RecipeLoad | Everyone |
| **LOAD** | RecipeLoad: the recipe into the editor | Everyone |
| **SAVE** | RecipeSave: the editor's values as the recipe | Maintenance and up |
| **DOWNLOAD TO PLC** | RecipeDownload: the editor's values to the PLC | Operator and up, sequence idle |
| **UPLOAD FROM PLC** | RecipeUpload: the PLC's values into the editor | Everyone |

| Button | Calls | Allowed for |
|---|---|---|
| **DELETE** | RecipeDelete, asking first | Maintenance and up |
| **EXPORT CSV…** | RecipeExport | Everyone |
| **IMPORT CSV…** | RecipeImport, saving under the name in the editor | Maintenance and up |
| **RENAME** | RecipeRename to the name typed above it | Maintenance and up |

Each button is an Action Script. Where access is limited, an **Enable** link (for example _AccessLevel >= 2000) blocks touch, and fill and text color links gray the button out. SAVE reports its result in the Last action box:

```
IF RecipeSave("Plasticizer Blends", RecipeName) THEN
  RecipeMsg = "Saved " + RecipeName;
ENDIF;
```

DOWNLOAD TO PLC is enabled by _AccessLevel >= 1000 AND Seq_Step == 0, so a recipe cannot change while a batch runs. DELETE passes 1 as the third argument, so the operator confirms first:

```
IF RecipeDelete("Plasticizer Blends", RecipeName, 1) THEN
  RecipeMsg = "Deleted " + RecipeName;
  RecipeName = "";
ENDIF;
```

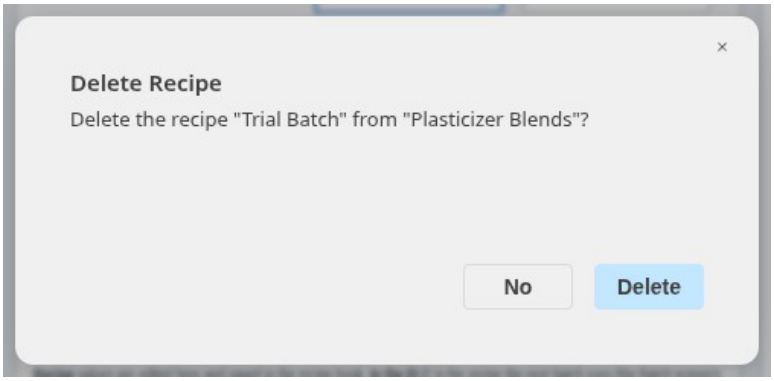



*Figure 31. RecipeDelete with confirm: the operator answers before anything is deleted*

## 11.3 The Recipe List object

**Recipe List**, in the HMI palette, shows a book's recipes while the application runs. Select it and open the Animation tab to set it up:

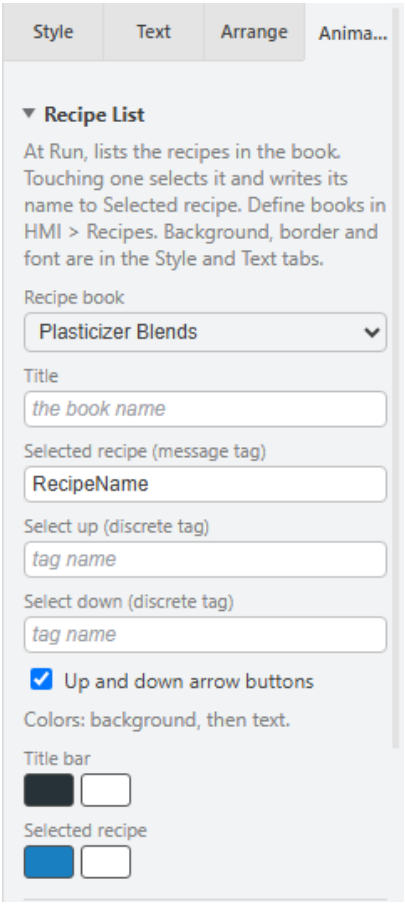



*Figure 32. The Recipe List settings of the Recipes screen's list*

*Table 32. Recipe List settings*

| Setting | Meaning |
|---|---|
| **Recipe book** | The book listed |
| **Title** | The heading; empty shows the book's name |
| **Selected recipe** | A message tag. The highlighted row follows its value, and touching a row writes the row's name to it |
| **Select up, Select down** | Discrete tags. Each change from 0 to 1 moves the selection up or down, for selection from PLC pushbuttons. With nothing selected, down selects the first recipe and up the last |
| **Up and down arrow buttons** | Draws arrow buttons in the heading that move the selection the same way |
| **Title bar** | The heading's background and text colors, in that order |

| Setting | Meaning |
|---|---|
| **Selected recipe colors** | The highlighted row's background and text colors |

The list is sorted by name and updates as soon as a recipe is saved, renamed or deleted. The rest of its look is the object's own, from the **Style** and **Text** tabs: **Fill** is the list's background, **Line** the border and its width, and the font family, size and color the rows' text.

## 11.4 Selecting a recipe in a window

ShowRecipeSelect opens a window listing a book's recipes, for screens without a Recipe List or for a popup choice. The operator touches a recipe, or uses the arrow buttons, then **Select**; the function returns the chosen name, or "" for **Cancel**. LiquidWeighHMI's SELECT RECIPE… button loads the chosen recipe into the editor:

```
RecipePick = ShowRecipeSelect("Plasticizer Blends");
IF RecipePick <> "" THEN
   RecipeName = RecipePick;
   RecipeLoad("Plasticizer Blends", RecipeName);
   RecipeMsg = "Loaded " + RecipeName + " into the editor";
ENDIF;
```

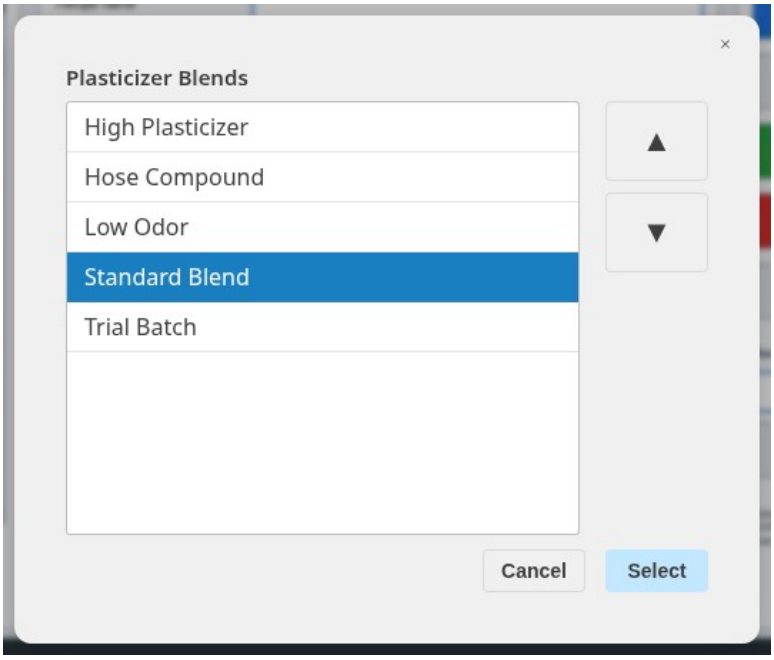



*Figure 33. The window opened by ShowRecipeSelect*

- The script waits at ShowRecipeSelect until the operator chooses, then continues with the result. It must therefore be a statement on its own or the whole right side of an assignment, as above; Validate reports it inside an IF condition or an expression.

- The window is centered. `ShowRecipeSelect(book, x, y, w, h)` places it instead, at x, y with width w and height h in screen pixels, scaled with the screen.
- Assigning the result to a separate tag, as above, keeps the current selection when the operator cancels.

## 11.5 When a recipe function fails

A recipe function that cannot do its job returns 0 and shows the **Recipe Error** window. The window names the call with its arguments and gives the reason, such as an empty recipe name, a recipe that does not exist, or PLC values with bad quality because the controller is not connected. Pressing SAVE with no recipe name gives:




*Figure 34. The Recipe Error window*

- The window opens after the script finishes, so the rest of the script still runs.
- Further failures before **OK** is pressed join the same window (up to 10, then "and n more"); a failure repeated by a While showing script is counted rather than repeated.
- The operator's own choices are not errors: Cancel in the select window, No when deleting and cancelling a file window return 0 or "" without one.
- Every failure is also written to the runtime log.

## 11.6 Where recipes are kept

The project holds the books and their starting recipes. Recipes saved while running (RecipeSave, RecipeRename, RecipeDelete, RecipeImport) are kept on the computer running the application, book by book: once a computer saves a recipe in a book, its own list replaces that book's starting recipes there.

*Table 33. Where saved recipes are kept*

| Running in | File |
|---|---|
| **Append HMI Studio (Run)** | recipes\<project>.json in the Studio's settings folder |
| **A published application** | %APPDATA%\<Product>\recipes\<Product>.json |
| **Append HMI Desktop, Append HMI Web** | recipes\<application>.json in their settings folder |

- **HMI > Clear Runtime Recipes** (with the Run stopped) forgets the recipes the Studio saved, so Run starts again from the starting recipes.
- In Append HMI Web all browsers share the server's recipes; one saved in one browser appears in the others within a few seconds.
- A recipe CSV file has a `#Recipe,<book>,<name>` line, then `Tag,Value` and one line per tag. Importing takes only the book's Save/Load tags and reports any others.

# 12 Trends

A **Trend** plots tag values over time. Drop one from the HMI palette onto a window and give it pens; while the application runs it shows the last few minutes, redrawn as the values change. Scripts move it: they pick the pen whose scale is shown, pause it, scroll back and forward through the history and return to live. The trend has no buttons of its own, so the screen's own buttons decide how operators use it.

## 12.1 The Trend object

Select the trend and open the Animation tab. Its settings come first:

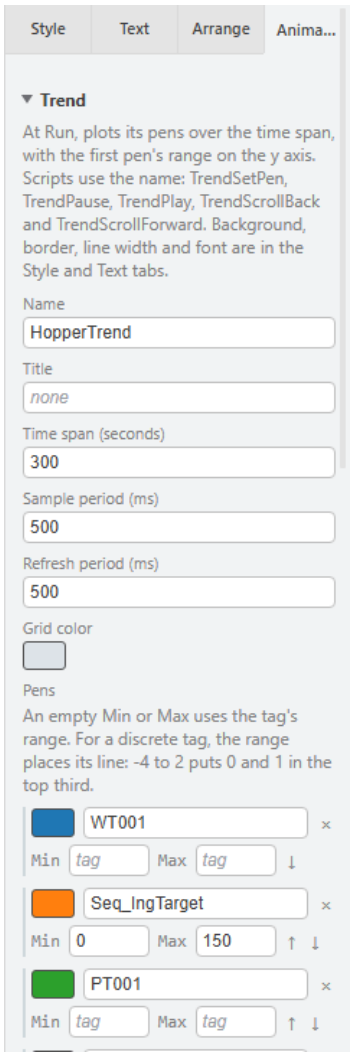



*Figure 35. The settings of LiquidWeighHMI's trend, with its first pens*

*Table 34. Trend settings*

| Setting | Where | Meaning |
|---|---|---|
| **Name** | Animation tab | The name scripts use. Each trend gets Trend1, Trend2, … and can be renamed; names are unique in the project, so a pasted or duplicated trend gets the next free one |
| **Title** | Animation tab | A heading above the plot; empty shows none |
| **Time span** | Animation tab | The time shown, in seconds: 10 s to 24 hours. 600 by default |
| **Sample period** | Animation tab | How often the pens are sampled, in milliseconds; at least 100 |
| **Refresh period** | Animation tab | How often the plot is redrawn while live, in milliseconds |
| **Grid color** | Animation tab | The grid lines and the plot's frame |
| **Pens** | Animation tab | Up to 8, each a tag, a color and a Min and Max (section 12.2) |
| **Fill, Line** | Style tab | The background and the border |

| Setting | Where | Meaning |
|---|---|---|
| **Line width** | Style tab | The width of every pen's line; the selected pen is 1 pixel thicker |
| **Font** | Text tab | The font, size and color of the title and the axis labels |

At Run the y axis on the left shows the **selected pen**: its range, and its name and units in its color. The first pen is selected when the application starts. The time axis along the bottom falls on whole seconds, minutes or hours. A gap in a line means there was no good value: bad quality, or the application was not running.

## 12.2 Pens and their ranges

A pen is an analog or discrete tag. Each pen is drawn against its own range, so lines with different units share the plot:

- **Min and Max** are the values at the bottom and top of the plot. Left empty, an analog pen uses the tag's Min EU and Max EU.
- **A discrete pen** is drawn as steps between 0 and 1. With no range of its own it uses -0.5 to 1.5, which puts its line in the middle half of the plot. Give each discrete pen its own range to stack them: -4 to 2 puts 0 and 1 in the top third, and 0 to 6 in the bottom third.

To put a discrete pen's steps between the fractions a and b of the plot's height (0 at the bottom, 1 at the top), use Min = -a / (b - a) and Max = (1 - a) / (b - a). LiquidWeighHMI's trend stacks its three valves in bands near the top, above the weight and pressure lines:

*Table 35. The pens of LiquidWeighHMI's trend*

| Pen | Tag | Min | Max | Drawn |
|---|---|---|---|---|
| **WT-001 hopper weight** | WT001 | (0) | (150) | lb, from the tag |
| **Ingredient target** | Seq_IngTarget | 0 | 150 | lb, on the weight's scale |
| **PT-001 hopper pressure** | PT001 | (0) | (100) | psi, from the tag |
| **LIT-100 tank 1 level** | LIT100 | (0) | (100) | %, from the tag |
| **SV-101 full feed** | SV101_Opened | -8.6 | 1.4 | Between 86 % and 96 % of the height |
| **SV-100 dribble** | SV100_Opened | -7.4 | 2.6 | Between 74 % and 84 % |
| **SV-401 discharge** | SV401_Opened | -6.2 | 3.8 | Between 62 % and 72 % |

A Min or Max in parentheses is left empty in the pen and taken from the tag.

## 12.3 The Trends screen

LiquidWeighHMI's **Trends** screen, on the TRENDS button of the navigation bar, follows the weigh hopper through a batch. In LiquidWeighHMISim press START on the Batch screen, then open TRENDS to see the hopper fill ingredient by ingredient, each in a full feed and then a dribble, and discharge.




*Figure 36. The Trends screen at Run in LiquidWeighHMISim, during a batch*

The buttons on the right are Action Scripts calling the trend functions, with the trend's name, HopperTrend, as a literal:

*Table 36. The Trends screen's buttons*

| Button | On Down script |
|---|---|
| **A pen (WT-001 HOPPER WEIGHT ...)** | TrendSetPen("HopperTrend", "WT001"); |
| **BACK** | TrendScrollBack("HopperTrend"); |
| **FORWARD** | TrendScrollForward("HopperTrend"); |
| **PAUSE** | TrendPause("HopperTrend"); |
| **LIVE** | TrendPlay("HopperTrend"); |

The color bar beside each pen button is a plain rectangle in the pen's color. The names can also come from message tags, for one set of buttons shared by several trends:

```
TrendSetPen(TrendName, PenTag);
```

The selected pen and a paused or scrolled position belong to the trend's name for the whole Run: they apply in every window that shows the trend and survive the window closing and opening again. A new Run starts live, with the first pen selected.

## 12.4 Scrolling through the history

BACK moves the plot back by half its span, so each press shows half of what was on screen before and half that is older. A paused trend shows **Paused** with the date and the times it covers in its top right corner. FORWARD moves toward now and stops there; LIVE returns at once.




*Figure 37. The trend scrolled back with BACK: paused, with the times it shows, and the hopper pressure pen selected. The empty part on the left is before the application started*

## 12.5 Recording and where the history is kept

Every pen of every trend is recorded from the moment the application starts, whether or not a window shows the trend, so a trend opened later already has its data.

- **Sampling:** each tag at the shortest sample period among the trends that have it.
- **What is kept:** a value when it changes, and once a minute while it does not. The last unchanged sample before a change is kept too, so a line stays flat up to the moment it moved.
- **Memory:** the last 24 hours. Scrolling back further reads the files on disk; long ranges are thinned to about 4000 points per pen, keeping the first, lowest, highest and last values of each stretch, so peaks still show.
- **Disk:** written every 5 seconds to one CSV file per day, `trends-YYYY-MM-DD.csv`, with the columns Time (milliseconds since 1970, UTC), Tag and Value (empty for a gap). Files are kept 7 days and at most 500 MB per application, the oldest going first.
- **After a restart** a trend shows what was recorded before it, with a gap while the application was not running.

*Table 37. Where the trend history is kept*

| Running in | Folder |
|---|---|
| **Append HMI Studio (Run)** | trends\<project> in the Studio's settings folder |
| **A published application** | %APPDATA%\<Product>\trends\<Product> |
| **Append HMI Desktop, Append HMI Web** | trends\<application> in their settings folder |

In Append HMI Web every browser runs the application and samples its trends, but one browser at a time writes the history on the server; when it closes or goes quiet for 15 seconds, another takes over. Browsers slow down the timers of a tab that is not shown, so a browser left in a background tab samples less often than its trends ask.

# 13 Databases

An application can read and write SQL databases from its scripts: log a batch as it finishes, keep a shift report, fetch the last records into a screen, look a part number up. Two kinds of database are supported, **Microsoft SQL Server** and **SQLite**, a single file that needs no server at all.

Four things are set up in **HMI > Database**, and the SQL functions work through them: **data sources** (the databases the application can open), **bind lists** (which tag goes in which column), **table templates** (the shape of a table the application creates for itself) and **queries** (statements kept in the project). LiquidWeighHMI uses all four to log every batch it completes.

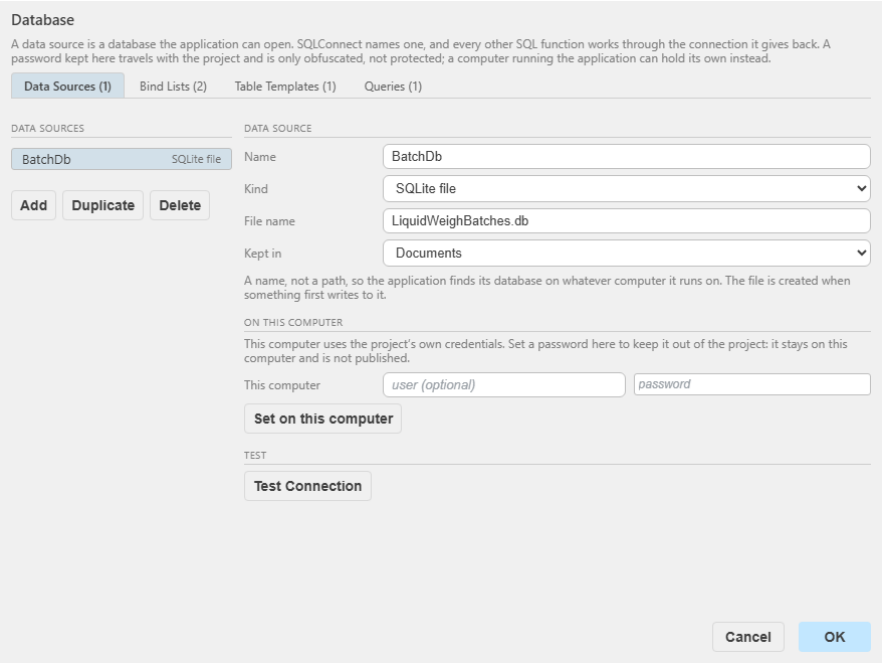



*Figure 38. HMI > Database: the BatchDb data source of LiquidWeighHMI*

## 13.1 Data sources

A data source is a database the application can open. SQLConnect names one and hands back a connection number that every other function uses.

*Table 38. Data source settings*

| Setting | Applies to | Meaning |
|---|---|---|
| **Name** | both | What SQLConnect names, up to 64 characters |
| **Kind** | both | Microsoft SQL Server, or a SQLite file |
| **Server, Port, Database, User, Password** | SQL Server | Where the server is and who to log in as. The port is 1433 unless set |
| **Instance name** | SQL Server | A named instance, in place of a port |
| **Encrypt the connection** | SQL Server | Off by default |
| **Accept any certificate** | SQL Server | On by default, which is what a server with a self-signed certificate needs |
| **Allow older TLS versions** | SQL Server | For an older server that offers only a TLS version this PC refuses by default |
| **Timeout (ms)** | both | How long a statement may take: 100 to 600,000. 15,000 unless set |
| **File name, Kept in** | SQLite | The file name, and whether it lives in the application data folder or in Documents |

**Test Connection** opens the data source and closes it again, and says what went wrong if it could not.

A SQLite data source names a **file and a folder, never a path**: an absolute path names one PC's account, and a published application runs on another PC, often as another user. The file is created the first time something writes to it. Documents is the place for a file somebody will open with another tool; the application data folder is the place for one that is the application's own business.

> **Note:** A password kept in the project is **obfuscated, not protected**: it is out of plain sight, and anyone with the project file and this product can read it back. Where that is not good enough, leave it empty and give each PC its own in the **On this computer** section of the same dialog, which is kept outside the project and wins over it. HMI > Clear Database Passwords forgets what this PC holds.

## 13.2 Bind lists

A bind list pairs a tag with a database column. It works in both directions: a **select writes the columns into the tags**, and an **insert or update reads the tags into the columns**. The order of the rows is the order of the columns.

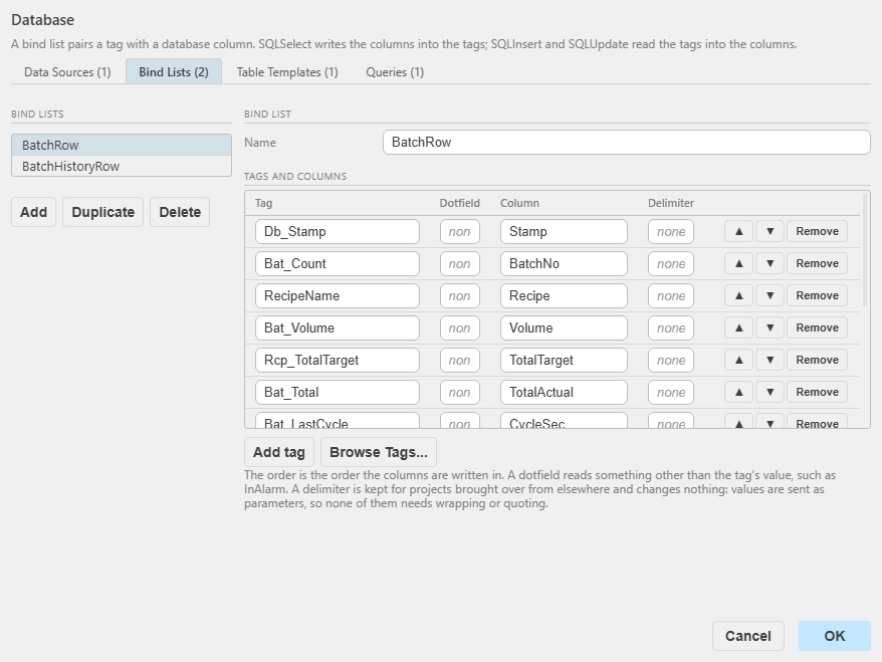



*Figure 39. The BatchRow bind list: each batch tag against its column*

A row can read a **dotfield** instead of the tag's value, such as `TimeDate` or `InAlarm`. A dotfield can only be read, so such a row is filled on an insert and skipped when rows are fetched.

- Values travel as **parameters**, never as text pasted into the statement, so nothing needs quoting and a tag holding an apostrophe cannot rewrite the statement.
- An insert sends the tags' **current** values. Every bind list's tags are watched from the moment the application starts, whether or not any window shows them, so a batch log records points nobody is watching.
- A value of bad quality is written as a null, and noted in the runtime log: one tag of twenty may be on a link that is down, and a record with a null in it still says what was known.
- A bind list names tags from the Tag Dictionary, so a system tag cannot be in one. To stamp a row with the time, use a tag's TimeDate dotfield, copy `_DateTime` into a message tag just before the insert, or let the database stamp it with GETDATE() or CURRENT_TIMESTAMP in a statement of your own.

## 13.3 Table templates and queries

A **table template** is the shape of a table, so an application can create the table it needs the first time it runs instead of somebody creating it by hand on every PC. Each column has a name, a type, a length or precision, whether it is indexed and whether it may be empty. SQLCreateTable creates the table and one index per column that asks for one. The types are the Studio's own, and each database gets whatever it wants:

*Table 39. Column types*

| Type | Microsoft SQL Server | SQLite |
|---|---|---|
| **Text** | NVARCHAR(length), 255 unless set | TEXT |
| **Whole number** | INT | INTEGER |
| **Number** | FLOAT | REAL |
| **Decimal** | DECIMAL(precision, scale), 18 and 2 unless set | NUMERIC |
| **On or off** | BIT | INTEGER |
| **Date, Time, Date and time** | DATE, TIME, DATETIME | TEXT (ISO 8601, which SQLite's date functions read) |

A table that is already there is left as it is, so the ordinary way to use this is to call SQLCreateTable in the application's On start script every time it runs: the first run makes the table and every later one finds it. Nothing is altered, so a table whose columns have since changed is not brought up to date.

A **query** is a statement kept in the project by name, loaded with SQLLoadStatement. It lives in the project rather than in a .sql file beside the installer, because a path names one PC's disk and a file somebody can edit is a file somebody can edit. A ? in a statement marks a parameter to be set before it runs.

## 13.4 The script functions

Every SQL function returns a **result code**: 0 when it worked, a negative number when it did not. SQLErrorMsg(code) turns a code into words for a message tag, and SQLErrorMsg(-1) gives the database's own words for the last failure. A failure also opens a **Database Error** window naming the call and the reason, so an operator is never left with a number alone.

```
Res = SQLInsert(ConnId, "BatchLog", "BatchRow");
IF Res <> 0 THEN
   Db_Status = SQLErrorMsg(Res);
ENDIF;
```

The functions that reach a database pause the script until it answers, like ShowRecipeSelect, so they must be a statement on their own or the whole right side of an assignment. Moving through rows already fetched, building a statement and setting a parameter are immediate.

*Table 40. Connecting*

| Function | What it does |
|---|---|
| **SQLConnect(ConnIdTag, "DataSource")** | Opens the data source and writes the connection number into ConnIdTag, an analog tag. Up to 8 connections at once; everything is closed when the application stops |
| **SQLDisconnect(ConnId)** | Closes the connection |

*Table 41. Reading rows*

| Function | What it does |
|---|---|
| **SQLSelect(ConnId, "Table", "BindList", "Where", "OrderBy")** | Fetches the bind list's columns. Either expression may be "". The rows are held by the application, so walking them costs nothing |
| **SQLFirst / SQLNext / SQLPrev / SQLLast (ConnId)** | Moves the cursor and writes that row into the bind list's tags. -5 when there is no such row, which is how a walk ends |
| **SQLGetRecord(ConnId, n)** | The row at number n, counting from 1 |
| **SQLNumRows(ConnId)** | How many rows were found. This one returns the count, not a result code |
| **SQLEnd(ConnId)** | Lets the rows go |

*Table 42. Writing rows and tables*

| Function | What it does |
|---|---|
| **SQLInsert(ConnId, "Table", "BindList")** | One new row from the current values of the bind list's tags |

| Function | What it does |
|---|---|
| **SQLUpdate(ConnId, "Table", "BindList", "Where")** | Sets those columns in the rows the where expression matches. An empty expression is refused, so an update cannot change every row by accident |
| **SQLUpdateCurrent(ConnId)** | Writes the bind list's tags back into the row the cursor is on |
| **SQLDelete(ConnId, "Table", "Where")** | The rows the expression matches; an empty one is refused |
| **SQLClearTable(ConnId, "Table")** | Every row, deliberately. The table stays |
| **SQLCreateTable(ConnId, "Table", "Template")** | A table shaped by one of the project's templates; a table that is already there is left alone |
| **SQLDropTable(ConnId, "Table")** | The table and everything in it |

A script can also build statements of its own. SQLSetStatement starts the connection's statement buffer and SQLAppendStatement adds to it (nothing is put between the pieces); SQLLoadStatement fills it from one of the project's queries; SQLPrepareStatement turns it into a statement with a number, so it can be run again with other values; and SQLExecute(ConnId, "BindList", StmtId) runs it, statement 0 being the buffer itself. A statement that returns rows fills them in through the bind list named there, and "" says the rows are not for tags.

Each ? in a prepared statement is given a value with the **SQLSetParam** family before it runs: SQLSetParamChar, SQLSetParamInt, SQLSetParamLong, SQLSetParamFloat, SQLSetParamDecimal, SQLSetParamDate, SQLSetParamTime, SQLSetParamDateTime and SQLSetParamNull, each taking the statement number, the parameter number and the value. SQLClearParam unsets one, and SQLClearStatement lets a statement go. For a loop that writes a row per pass, SQLInsertPrepare, SQLInsertExecute and SQLInsertEnd do the same for an insert built from a bind list.

SQLTransact(ConnId), SQLCommit(ConnId) and SQLRollback(ConnId) put several writes together: everything between the first and the second reaches the tables together, or not at all. An application that stops with a transaction open rolls it back.

## 13.5 The batch log in LiquidWeighHMI

LiquidWeighHMI writes a record of every batch to a SQLite file in the Documents folder of the PC that runs it, and shows them again on its Batch History screen. Nothing has to be installed or set up for it: the application makes its own file and its own table.

Its **On start** application script opens the database and makes sure the table is there:

```
DIM res AS Analog;

res = SQLConnect(Db_ConnId, "BatchDb");

IF res <> 0 THEN
   Db_Status = SQLErrorMsg(res);
ELSE
   res = SQLCreateTable(Db_ConnId, "BatchLog", "BatchLogTable");
   Db_Status = "Logging batches to BatchDb";
ENDIF;
```

Its **While running** script watches the controller's batch counter. When it moves on, a batch has finished: the script stamps the row from the clock, notes who was logged in and writes it through the BatchRow bind list.

```
IF Bat_Count <> Db_LastCount AND Db_ConnId > 0 THEN
   Db_LastCount = Bat_Count;
   Db_Stamp = _DateTime;
   Db_Operator = _Username;
   res = SQLInsert(Db_ConnId, "BatchLog", "BatchRow");

   IF res == 0 THEN
      Db_Status = "Batch " + StringFromIntg(Bat_Count) + " logged at " + _TimeString;
   ELSE
      Db_Status = SQLErrorMsg(res);
   ENDIF;
ENDIF;
```

The **Batch History** screen, opened by the BATCH HISTORY button on the Batch screen, runs the project's RecentBatches query when it opens and shows one batch at a time. NEWEST, NEWER, OLDER and OLDEST move the cursor through the rows the query found, and each row lands in the tags of the BatchHistoryRow bind list, which is what the boxes show. REFRESH runs the query again.

```
res = SQLLoadStatement(Db_ConnId, "RecentBatches");

IF res == 0 THEN
   res = SQLExecute(Db_ConnId, "BatchHistoryRow", 0);
```

```
ENDIF;

Db_Rows = SQLNumRows(Db_ConnId);
res = SQLFirst(Db_ConnId);
```

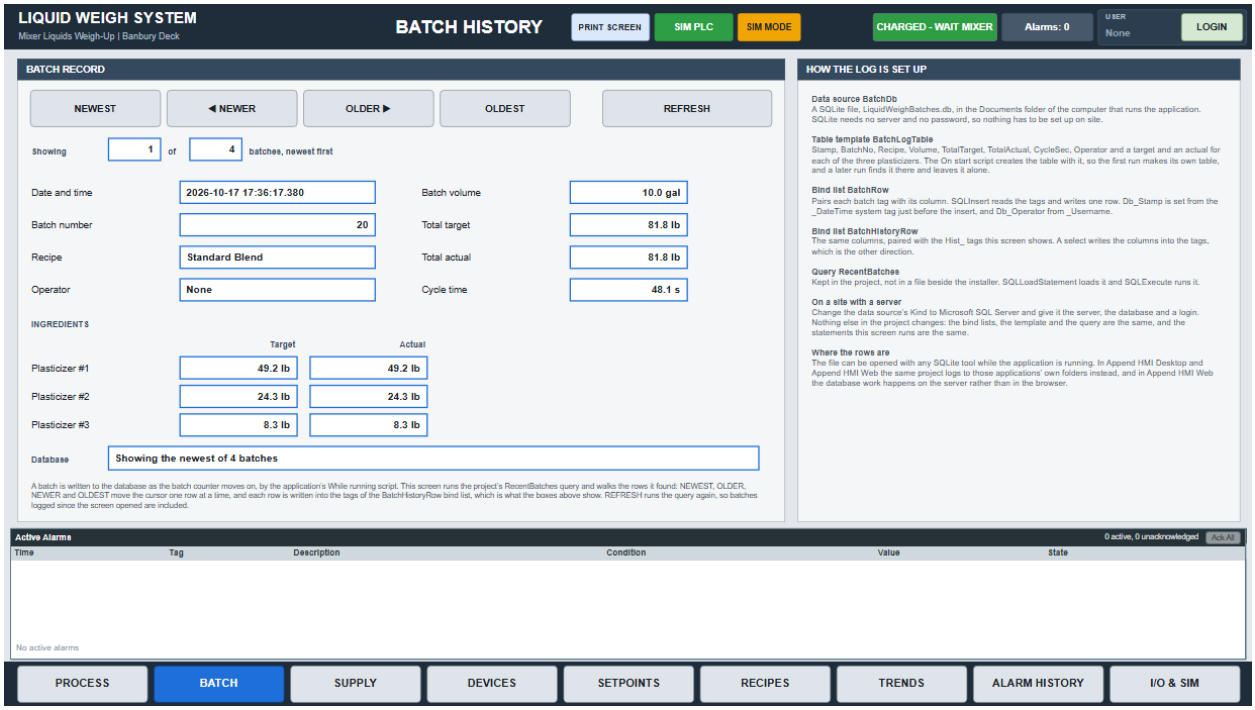



*Figure 40. The Batch History screen of LiquidWeighHMISim, showing the newest of the batches it has logged*

LiquidWeighHMISim logs the same way and needs neither a PLC nor a server, so it is the quickest way to see the whole thing work: open it, press START or AUTO CYCLE, and each batch appears on the Batch History screen as it finishes. The file can be opened with any SQLite tool while the application is running.

## 13.6 Where the rows are kept

Only the application itself talks to the database: the screens never do, and a password never reaches them. A SQLite file lives beside the application's other data:

*Table 43. Where a SQLite file is kept*

| Running in | Folder |
|---|---|
| **Append HMI Studio (Run)** | databases in the Studio's settings folder, or Documents |
| **A published application** | %APPDATA%\<Product>\databases, or Documents |
| **Append HMI Desktop, Append HMI Web** | databases in their settings folder, or Documents |

In Append HMI Web the application runs in the browser but the database work happens on the server, so the browser needs no database access and the credentials never leave the server. Each browser gets

its own connections, as it gets its own login. Several browsers writing one table is what a database is for; for more than a handful, a server database is a better answer than a SQLite file.

> **Note:** Microsoft SQL Server's datetime column keeps about 1/300 of a second, so a stamp written as .969 reads back as .970. Where the exact millisecond matters, make the column datetime2 with a statement of your own.

# 14 List boxes

A **List Box** shows a list of text items the operator picks from. Unlike the Alarm List, the Recipe List and the Trend, which each show something the application already keeps, a list box holds whatever the scripts put in it: the batches waiting to run, the lots a query returned, the faults logged since the shift started. Whatever the operator touches is written to a message tag, and the rest of the screen follows that tag like any other.

## 14.1 The List Box object

Drop a **List Box** from the HMI palette onto a window, select it and open the Animation tab. Its settings come first:

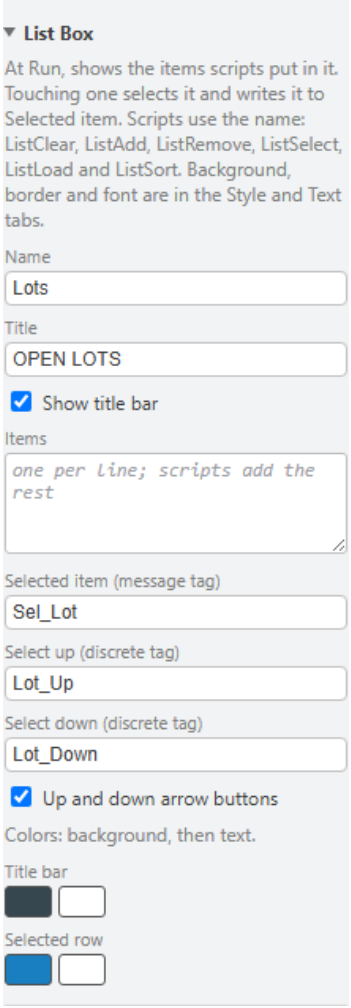



*Figure 41. The settings of a list box*

*Table 44. List Box settings*

| Setting | Where | Meaning |
|---|---|---|
| **Name** | Animation tab | The name scripts use. Each list gets List1, List2, ... and can be renamed; names are unique in the project, so a pasted or duplicated list gets the next free one |
| **Title** | Animation tab | The heading, when the title bar is shown; empty uses the list's name |
| **Show title bar** | Animation tab | Draws the heading, and holds the arrow buttons. Off by default: a plain list of items is what most screens want |
| **Items** | Animation tab | Starting items, one per line. The list holds these when the application starts, before any script has run |
| **Selected item** | Animation tab | A message tag the selection is written to |
| **Select up, Select down** | Animation tab | Discrete tags: a change from 0 to 1 moves the selection one item, so a pair of buttons or physical inputs can drive the list |

| Setting | Where | Meaning |
|---|---|---|
| **Up and down arrow buttons** | Animation tab | Two buttons in the title bar that do the same |
| **Title bar background, text** | Animation tab | The heading's colors |
| **Selected row background, text** | Animation tab | The selected item's colors |
| **Fill, Line, Line width** | Style tab | The background behind the rows and the border |
| **Font** | Text tab | The font, size and color of the rows |

A color left unset keeps the built-in look, so setting one of them leaves the rest as they were. The rows scroll when there are more than fit, and the selected row is always brought back into view: after a sort, after an item above it is removed, and when a script selects something off screen.

Touching a row selects it and writes it to the **Selected item** tag at once, so a value display, a button's Enable when or a script sees the choice immediately. With nothing selected, Select down picks the first item and Select up the last.

## 14.2 Filling and working the list from scripts

Six functions work a list by its name, as text: a literal in quotes or a message tag holding the name. They sit with the other script functions in section 9.3.

*Table 45. List functions (scripts only)*

| Function | What it does |
|---|---|
| **ListClear(list)** | Removes every item and clears the selection |
| **ListAdd(list, item)** | Adds the item to the bottom of the list |
| **ListLoad(list, text)** | Adds the comma-separated items in text to the bottom |
| **ListRemove(list, item)** | Removes the item |
| **ListSelect(list, item)** | Selects the item |
| **ListSort(list, order)** | Sorts the items alphabetically: "ASC" or "A", "DESC" or "D" |

```
ListClear("Batches");
ListLoad("Batches", "B-1024, B-1025, B-1026");
ListAdd("Batches", NewBatch);
ListSort("Batches", "ASC");
ListSelect("Batches", NewBatch);
```

- **Items are text.** An item is trimmed as it goes in, and an empty one is refused. Duplicates are allowed: the script decides what the list holds.
- **Finding an item ignores case**, so ListRemove("Flavors", "vanilla") removes Vanilla. ListRemove and ListSelect **do nothing and still succeed** when the list has not got the item, so a script that tidies up after the operator raises no errors. ListRemove takes the first match when an item appears twice.
- **ListLoad splits on commas and trims**, dropping empty entries: " Red , Green ,, Blue, " loads Red, Green and Blue. There is no quoting, so an item that contains a comma goes in with ListAdd.
- **Sorting ignores case** and keeps the selection: the selected item stays selected, and the list scrolls so it is still in view.
- **Removing the selected item clears the selection**, and so does ListClear; the Selected item tag is set to empty.

Each function returns 1, or 0 with a List Error window naming the call and the reason, such as a list that does not exist, a sort order that is not ASC, DESC, A or D, or an item too many: a list holds at most 5000. Validate reports a quoted list name that no screen has, and a bare tag where a name belongs.

## 14.3 The list belongs to the application, not to a window

Every list box exists from the moment the application starts, whether or not a window shows it. So an **On start** application script (section 9.1) can fill a list the operator opens later; two windows showing the same list show the same items and the same selection; and closing a window and opening it again leaves the list as it was. The Selected item tag follows the selection even with no window open, and is written only when the selection moves, so filling a list does not write the tag item by item.

Items are **not** kept between runs. Each run starts with the Items box's own lines and whatever the scripts add. To hold a choice across runs, write the Selected item tag to a retentive memory tag (section 5.1) and select it again in an On start script.

## 14.4 A list filled from a database

A list box and the database functions (chapter 13) go together: a query's rows become the items the operator picks from. This On show script fills a list named Lots with the open lots:

```
DIM Res AS Analog;

ListClear("Lots");
Res = SQLSelect(ConnId, "Lots", "LotRow", "Status = 'Open'", "LotId");
Res = SQLFirst(ConnId);

WHILE Res == 0
   ListAdd("Lots", LotId);
   Res = SQLNext(ConnId);
ENDWHILE;

Res = SQLEnd(ConnId);
```

Each move writes the row into the bind list's tags, so LotId holds the value of the row the cursor is on. The operator then picks a lot, its name goes to the Selected item tag, and a Touch Pushbutton loads it.

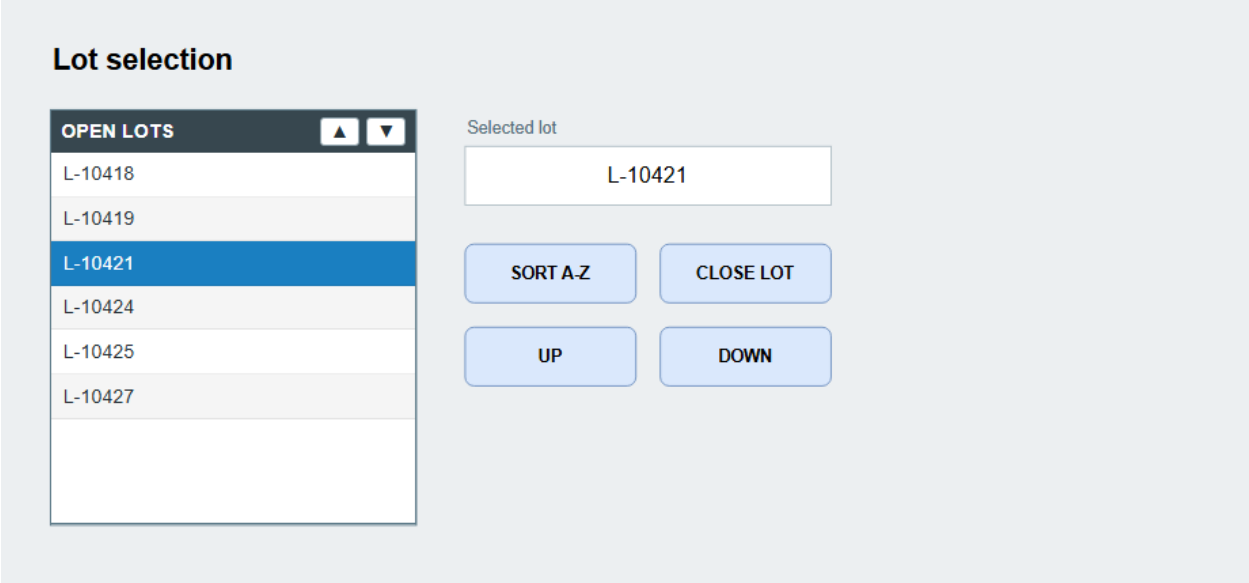



*Figure 42. A list box at Run: the items a script loaded, with one selected*

# 15 Testing

## 15.1 Validate

**HMI > Validate Expressions** checks every link, script and PLC address and lists any problem with the page and object; click a problem to go to it. Validate before every Run and Publish.

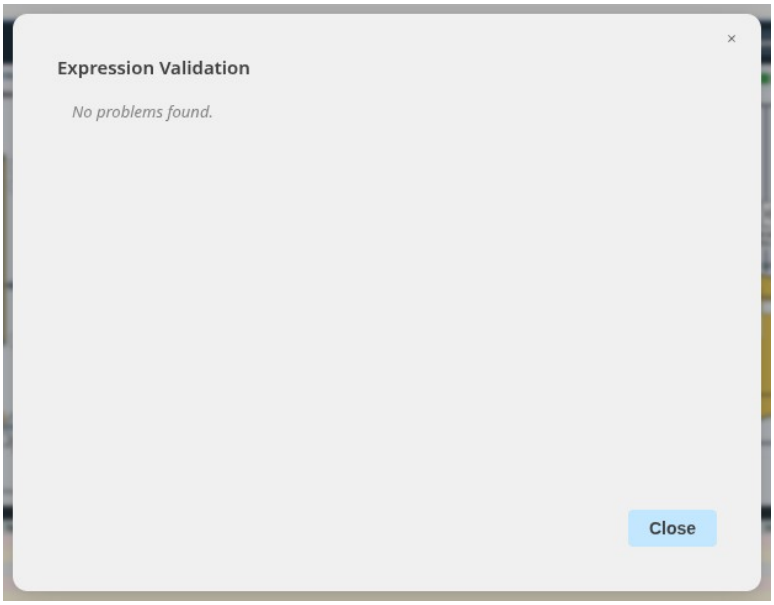



*Figure 43. Validation of LiquidWeighHMI: no problems found*

## 15.2 Run

**HMI > Run** (F5) runs the application inside the Studio: the drawing is locked, the start-up windows open, and the application connects to its PLCs. A green RUNNING banner shows that the screen is live and that touching it operates real equipment. **HMI > Stop** returns to editing.

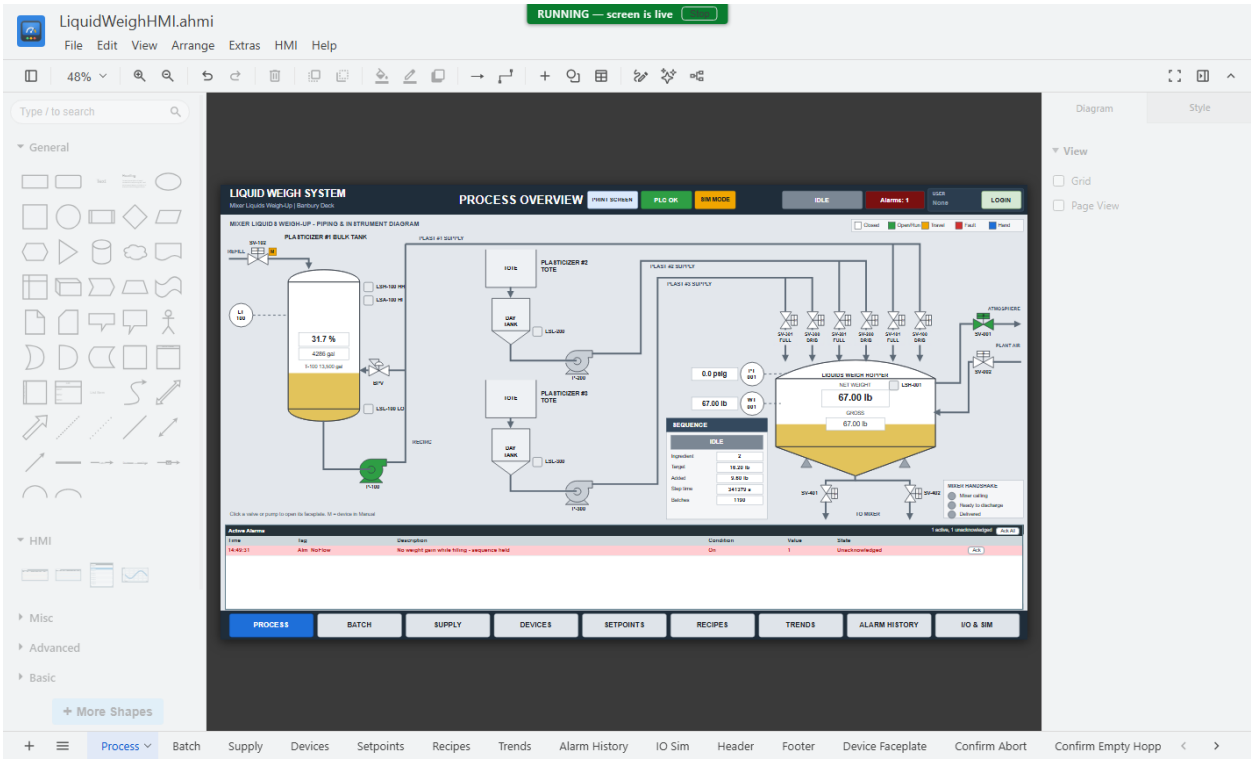



*Figure 44. LiquidWeighHMI running in the Studio*

The screens below were captured while LiquidWeighHMI ran against its controller, between batches. The header and the navigation bar above and below them are the Header and Footer windows (section 6.2), open over every screen: the system name, the screen title, **PRINT SCREEN** (saves the screen as a PDF, section 9.6), the PLC and simulation status, E-STOP, the sequence state, the number of active alarms, and the user with **LOGIN**.

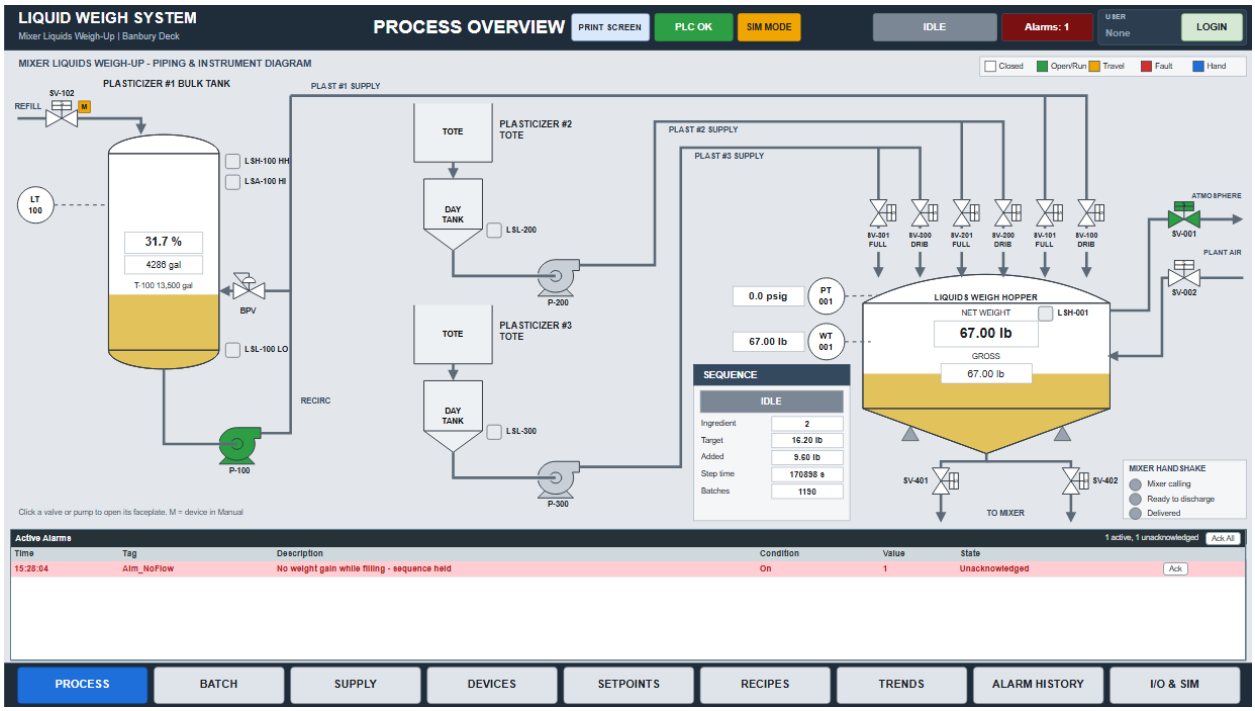



*Figure 45. Process screen: tank and hopper levels, valve states, sequence and active alarms*

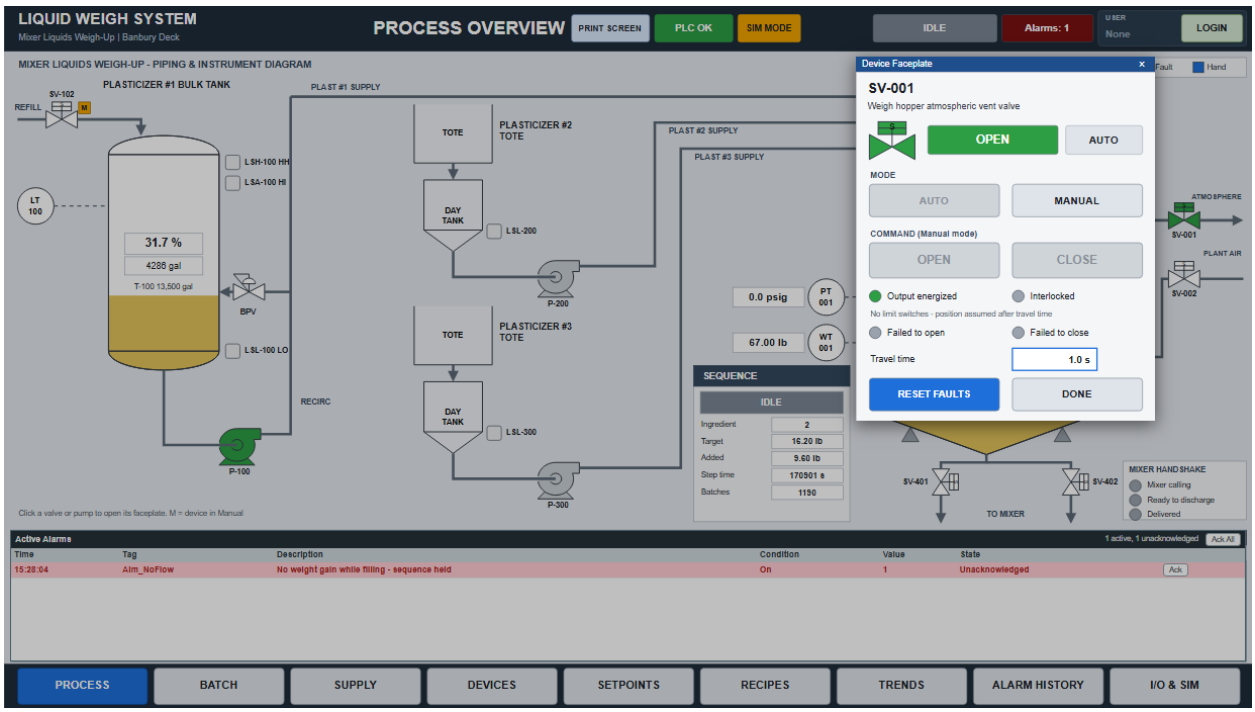



*Figure 46. The Device Faceplate showing SV-001: mode, commands, interlocks and fault reset*

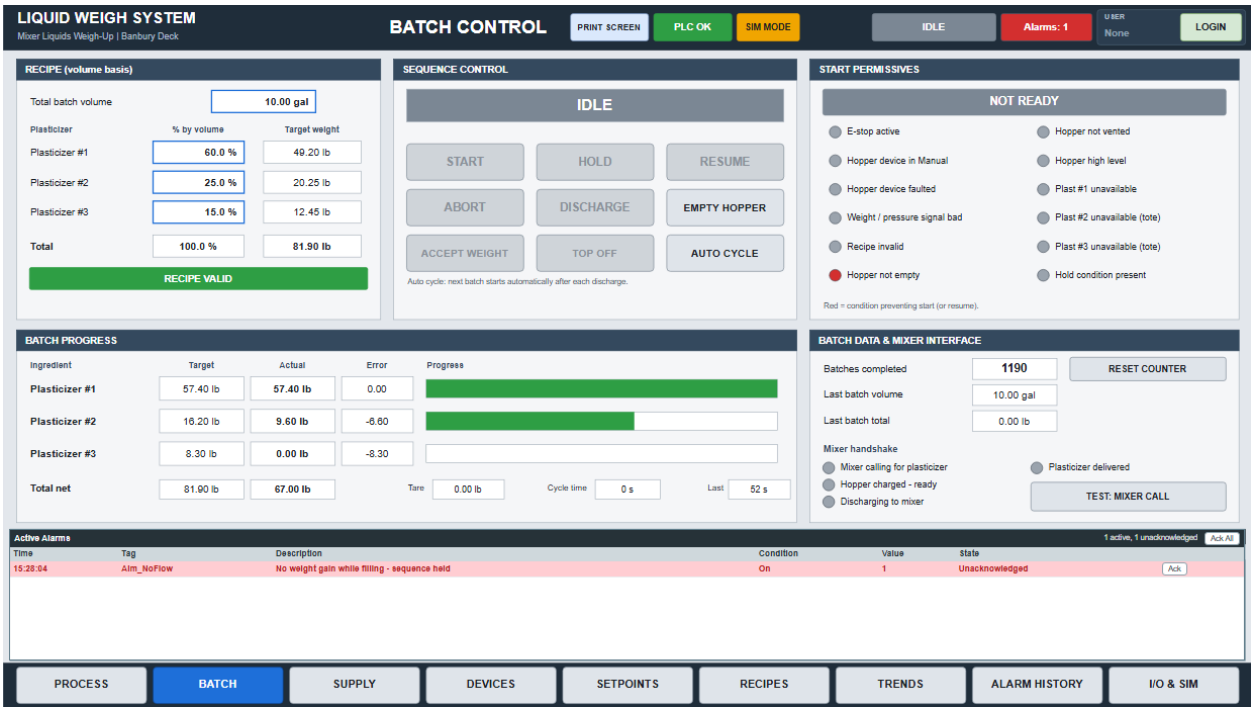



*Figure 47. Batch screen: recipe, sequence control, start permissives and batch progress*

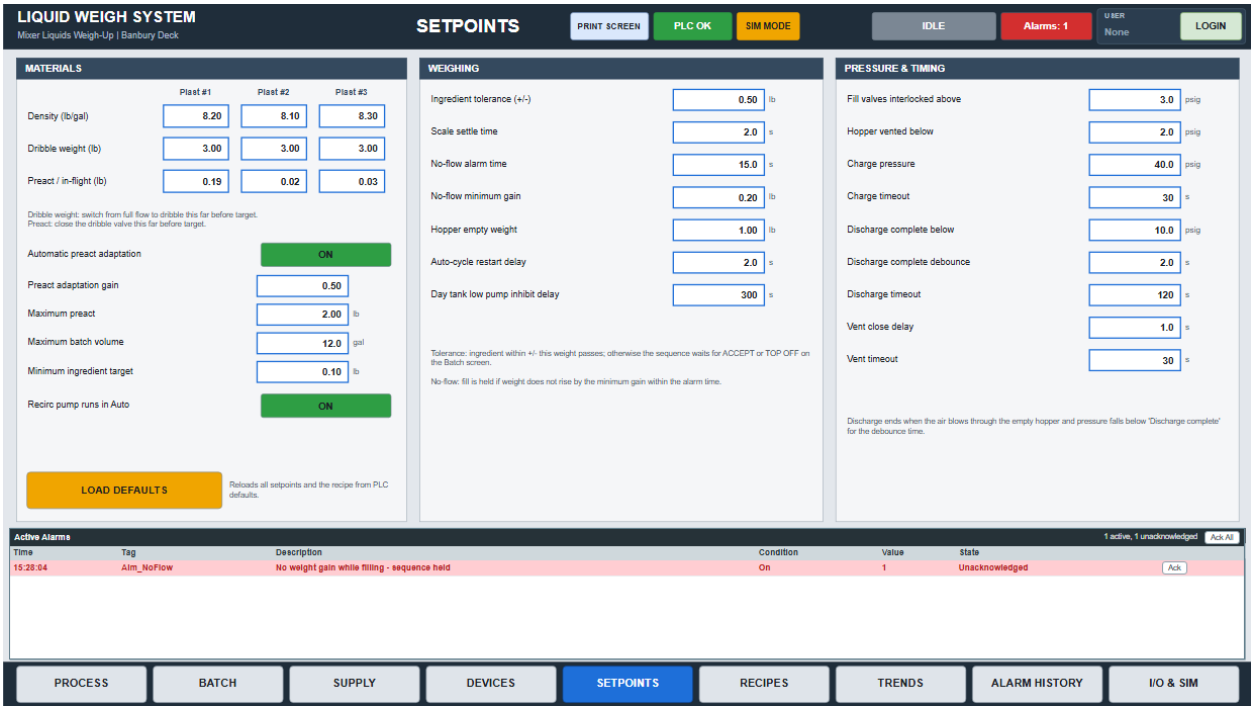



*Figure 48. Setpoints screen: materials, weighing, pressure and timing*

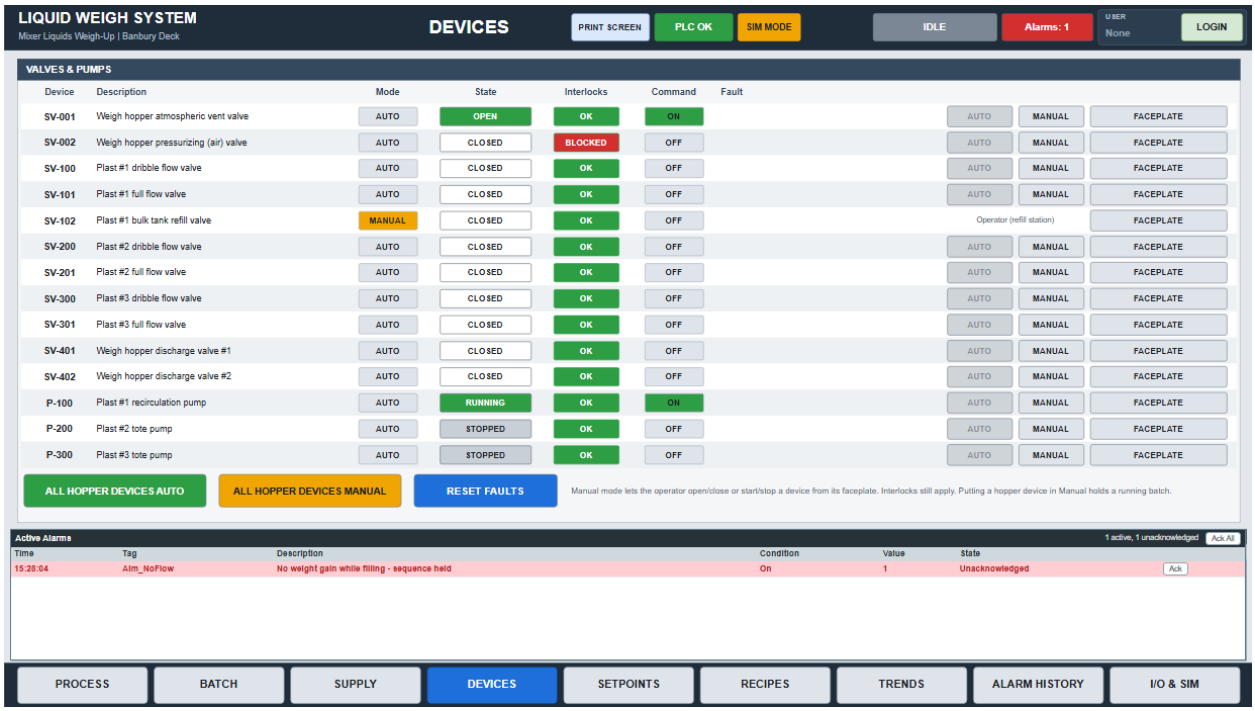



*Figure 49. Devices screen: every valve and pump with mode, state and faceplate*

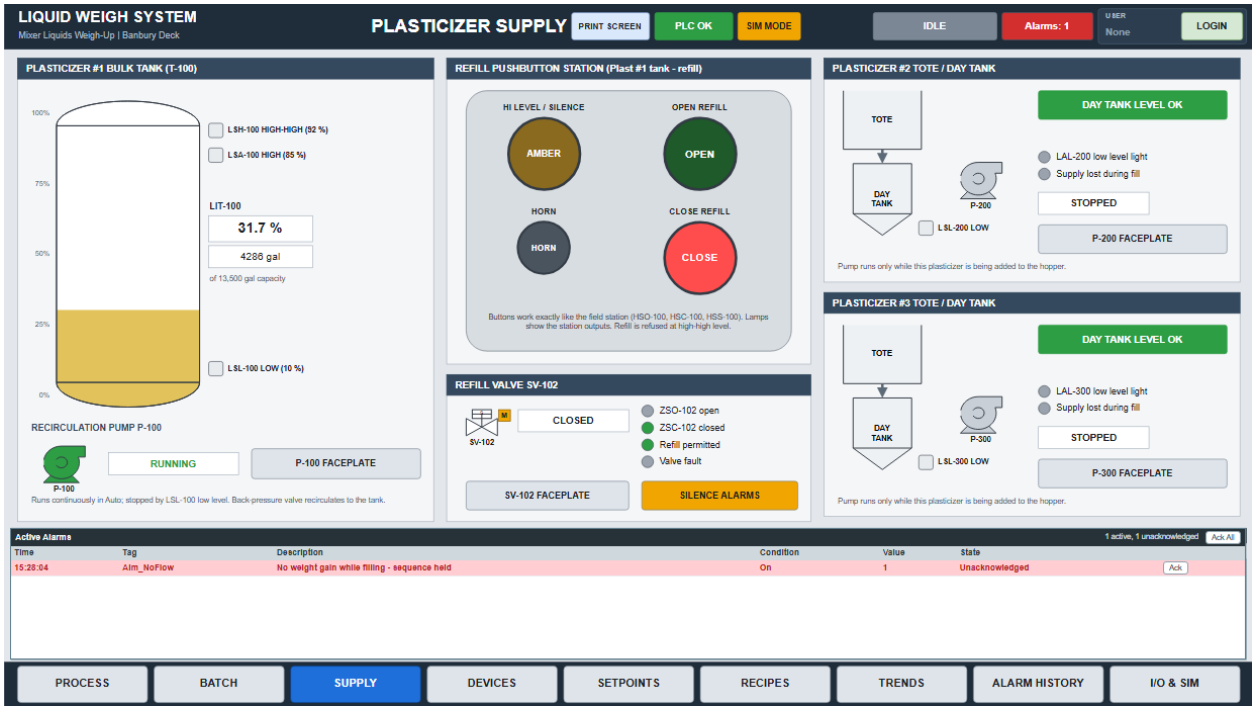



*Figure 50. Supply screen: plasticizer tank, refill station and totes*

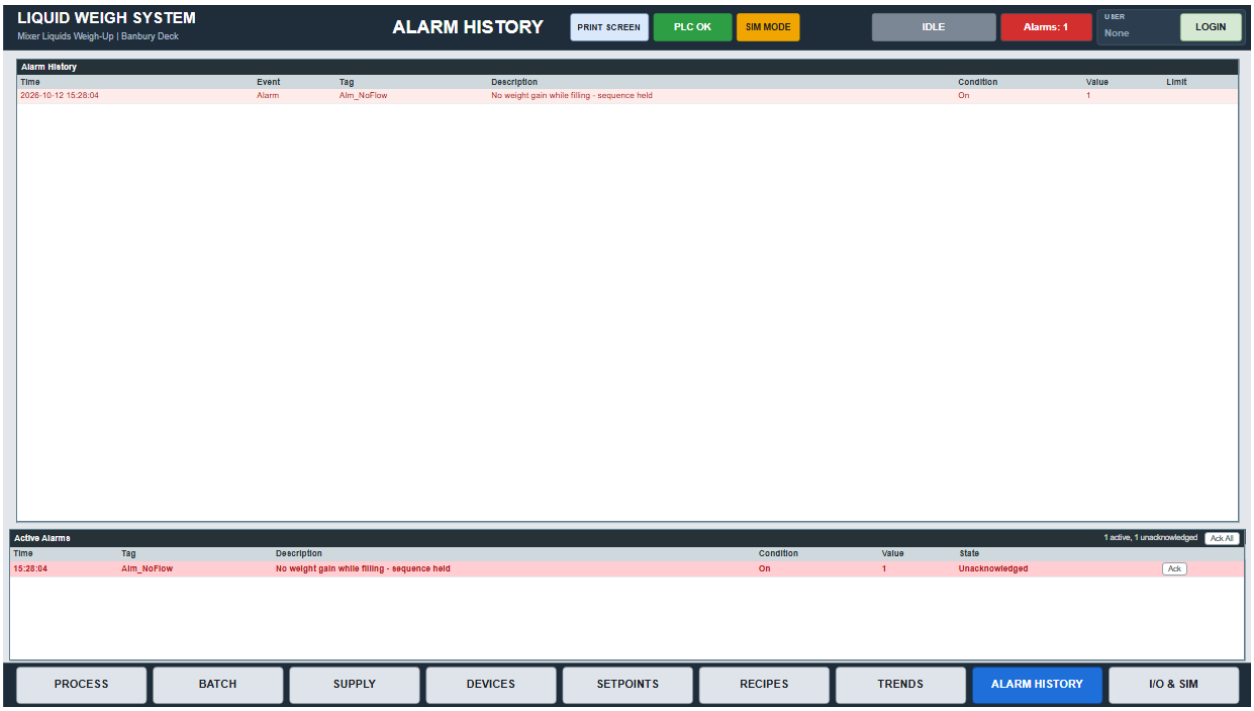



*Figure 51. Alarm History screen*

**Note:** HMI > Runtime Log lists connections, device errors, writes and script errors from the run.

# 16 Deploying

An application can reach the plant floor in three ways.

*Table 46. Deployment options*

| Option | What the target gets | Best for |
|---|---|---|
| **HMI > Publish** | A Windows installer of this one application, with its own name and icon, starting straight into it | A dedicated, locked-down operator station |
| **Append HMI Desktop** | One runner installed once; runs any .ahmi file, from shortcuts or at log-in | Stations whose projects change often; Linux |
| **Append HMI Web** | A web server; operators open the application in any browser | Viewing and operating from tablets, laptops and control-room PCs |

## 16.1 Publish

**HMI > Publish** builds a Windows installer from the open project. Give the product name, version and publisher, choose an icon, whether it installs for all users, and whether it adds a desktop shortcut or starts with Windows. Run the Setup.exe on the target PC; the application then starts full screen as set in Application Settings.

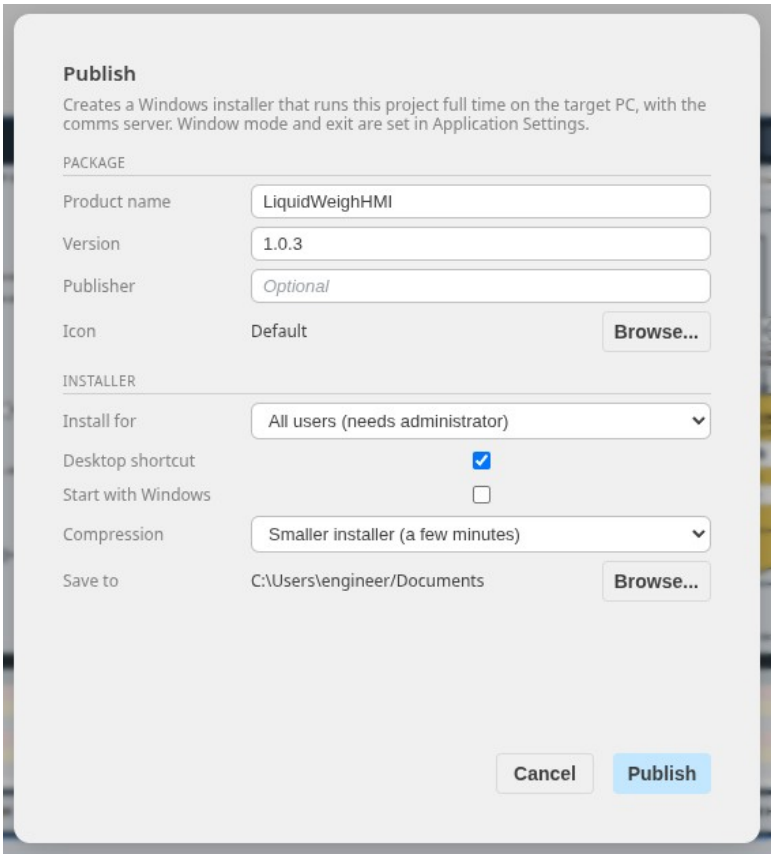



*Figure 52. The Publish dialog*

## 16.2 Append HMI Desktop

Append HMI Desktop runs .ahmi files without the editor. Choose an application, then **Run now**, create a desktop shortcut or menu entry, or tick **Run this application automatically when I log in**. From the command line: append-hmi-desktop LiquidWeighHMI.ahmi.

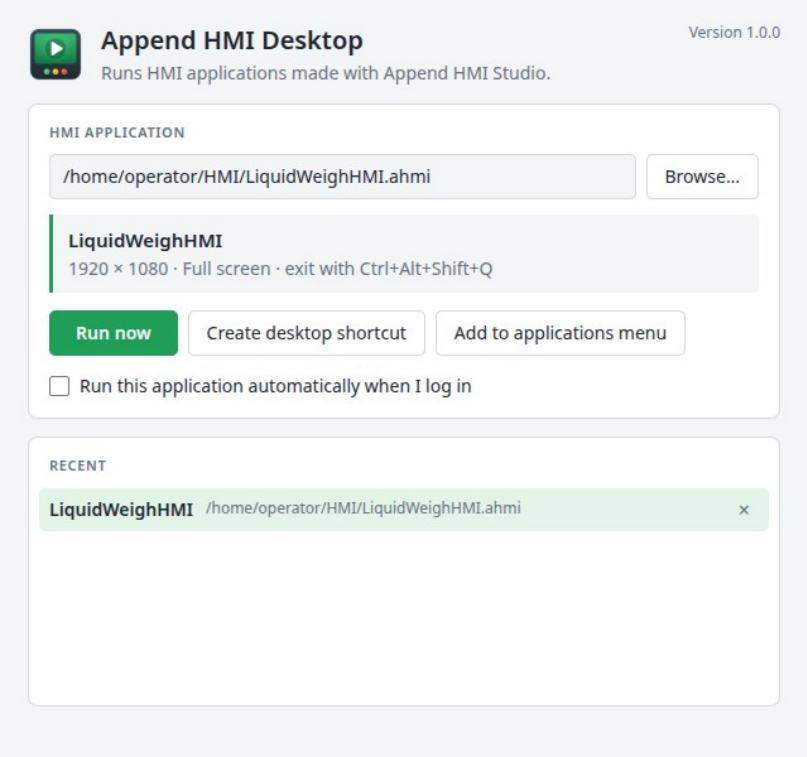



*Figure 53. The Append HMI Desktop launcher with LiquidWeighHMI selected*

## 16.3 Append HMI Web

Append HMI Web serves an application to web browsers on the plant network. Choose the application and a port (8480 by default), then **Start web server**; the launcher shows the link to share, with a **Copy** button. If the port is already in use on the PC (by another web server, or another application being served), choose another. In the browser, the view button in the corner fits the screen to the window, maximizes it, or shows it at its original size with scroll bars.

The **Running** list shows every Append HMI Web server on the PC, including ones started in the background from the command line (--headless). Each has **Restart**, which starts it again with the same settings and reads the project file afresh (do this after saving changes in Studio; browsers reload by themselves), and **Stop**.

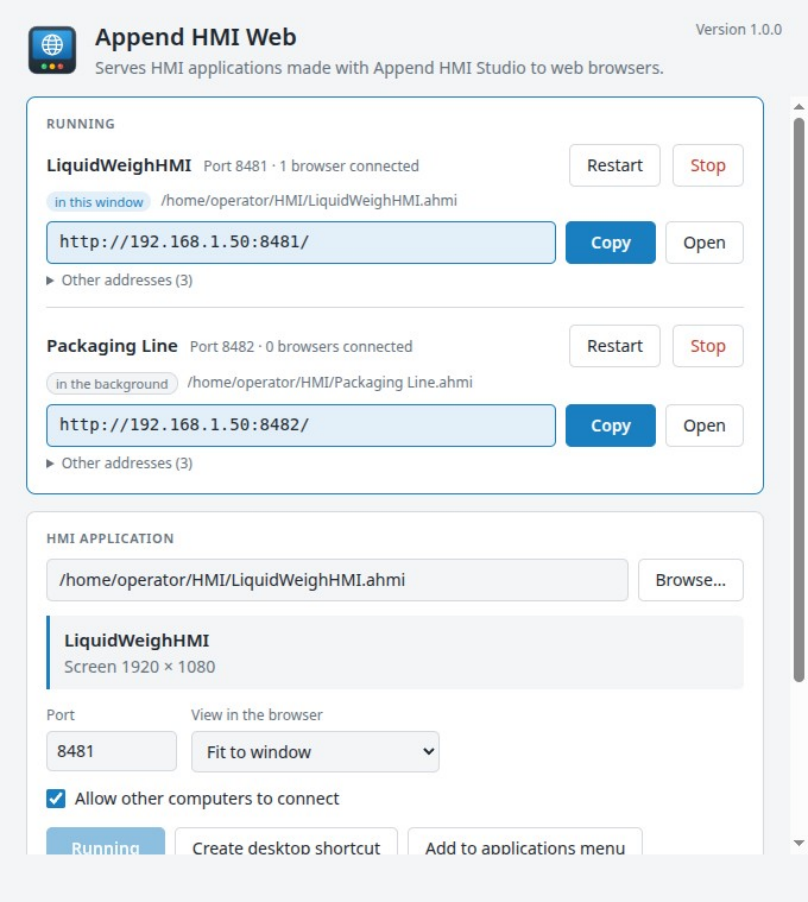



*Figure 54. The Append HMI Web launcher: LiquidWeighHMI served on port 8481 to one browser, and a second application running in the background*

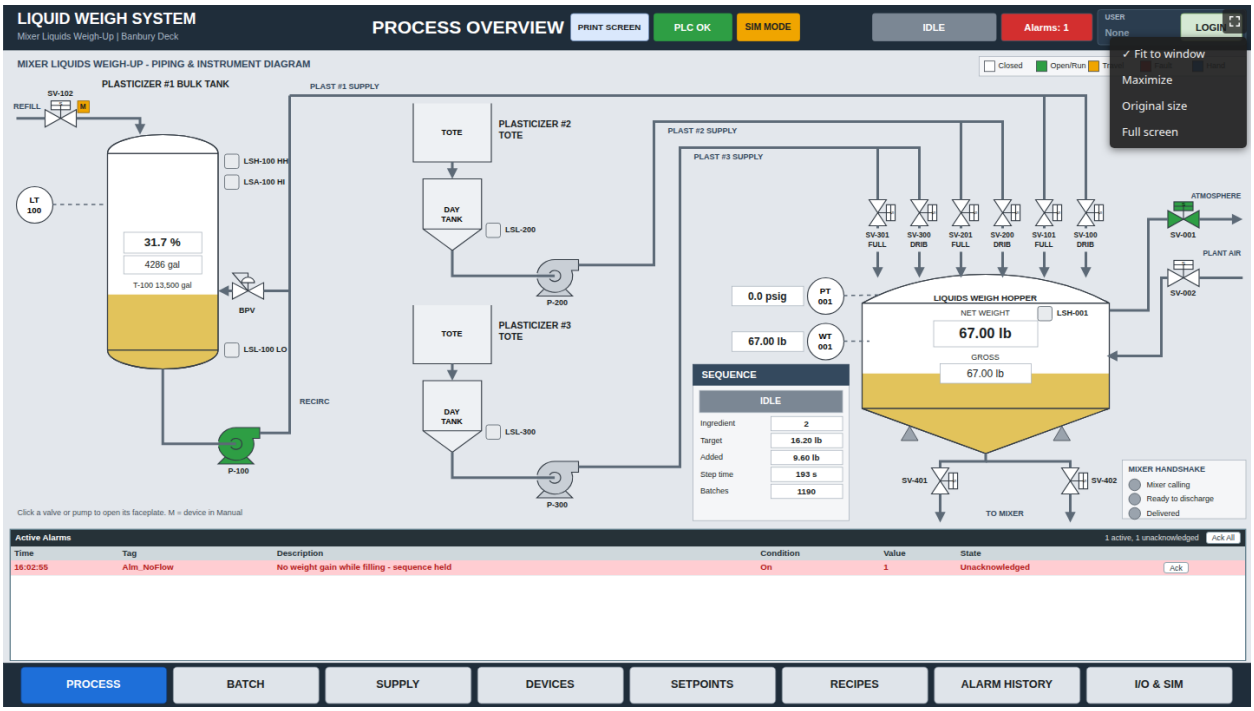



*Figure 55. LiquidWeighHMI in a web browser, with the view menu open*

# 17 Tips and troubleshooting

*Table 47. Common problems*

| Symptom | Cause and remedy |
|---|---|
| **Values show ####** | The tag has bad quality: the PLC is not connected or the address is wrong. Check HMI > Runtime Log and Test Connection in HMI > Devices |
| **PLC COMM FAIL in the header** | The device is unreachable. Check the IP address, slot and network |
| **A button does nothing** | An Enable or Pushbutton "Enable when" condition is false, or nobody with the access level is logged in |
| **Validate reports an expression error** | Expressions take no ";" (only script statements do); tag names must exist in the Tag Dictionary |
| **A popup opens in the wrong place** | Its objects must be drawn at the window's position on its page (Window Properties > Left, Top) |
| **Retentive values are not what you expect** | Use HMI > Clear Retentive Values to start from the initial values again |
| **A recipe button does nothing** | Its Enable condition is false (access level, or the sequence is running), or the function failed: the Recipe Error window and HMI > Runtime Log give the reason |
| **Run shows old recipes** | Recipes saved on this computer replace the starting ones. Use HMI > Clear Runtime Recipes to start from the project's recipes again |

| Symptom | Cause and remedy |
|---|---|
| **A trend shows no line, or a gap** | The pen's tag has no good value (the PLC may not be connected), or the application was not running then. A pen whose Min and Max leave its values off the plot draws nothing visible |
| **A trend button does nothing** | The Function Error window names the call and the reason: usually a trend or pen name that does not match. Validate checks quoted names |

Build a symbol once, with its links, group it, and copy it: links travel with the copy. The Animation tab can also copy links from one object and paste them onto others.

# Appendix A  Command-line automation

Append HMI Studio can build, check, render and publish applications without its window, for scripting and for AI agents (section 1.1):

*Table 48. Command-line modes*

| Command | Does |
|---|---|
| **--hmi-build spec.json -o app.ahmi** | Builds a project from a JSON description |
| **--hmi-check app.ahmi** | Validates it (exit code 2 when problems are found) |
| **--hmi-render app.ahmi -o folder** | Renders every page to PNG |
| **--hmi-dump app.ahmi -o spec.json** | Writes a project back out as JSON |
| **--hmi-publish app.ahmi --product "Name" --app-version 1.0.0 -o folder** | Builds the Windows installer |

Exit code 0 means no problems, 2 means the project was written but has problems (each printed as a PROBLEM line), and 1 means it failed. The JSON format is documented in doc/HMI_AUTOMATION.md in the Append HMI Studio repository, and the Claude Code skill in .claude/skills/append-hmi-studio teaches an agent the whole workflow.